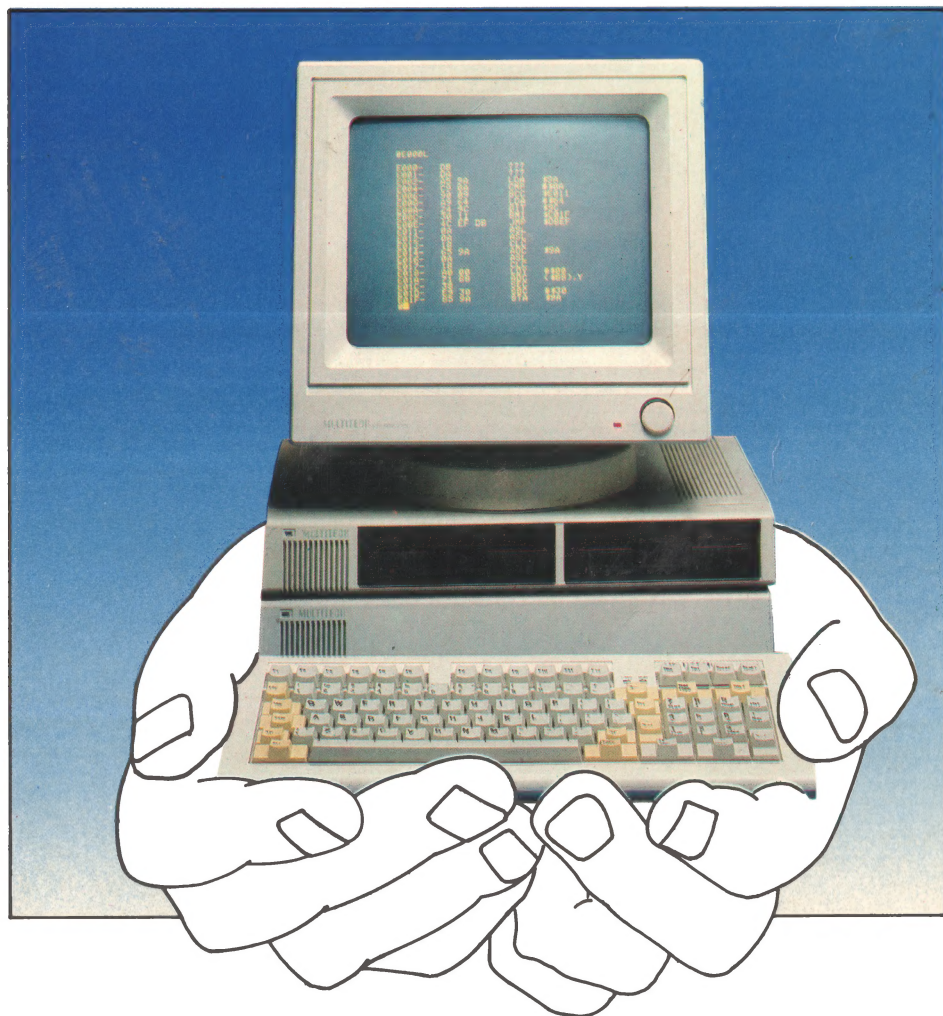


 MPF-III
Reference Manual



MPF-III
Reference Manual

COPYRIGHT

Copyright © 1983 by MULTITECH INDUSTRIAL CORP. All rights reserved. No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of MULTITECH INDUSTRIAL CORP.

DISCLAIMER

MULTITECH INDUSTRIAL CORP. makes no representations or warranties, either express or implied, with respect to the contents hereof and specifically disclaims any warranties or merchantability or fitness for any particular purpose. MULTITECH INDUSTRIAL CORP. software described in this manual is sold or licensed "as is" Should the programs prove defective following their purchase, the buyer (and not MULTITECH INDUSTRIAL CORP., its distributor, or its dealer) assumes the entire cost of all necessary servicing, repair, and any incidental or consequential damages resulting from any defect in the software. Further, MULTITECH INDUSTRIAL CORP. reserves the right to revise this publication and to make changes from time to time in the content hereof without obligation of MULTITECH INDUSTRIAL CORP. to notify any person of such revision or changes.



Multitech INDUSTRIAL
CORP.

OFFICE 315 FU HSIN N. RD., TAIPEI, TAIWAN, R.O.C.
FACTORY / 1 INDUSTRIAL E. RD., III HSINCHU SCIENCE-BASED
INDUSTRIAL PARK, HSINCHU, TAIWAN, R.O.C.

Preface

This is the Technical Reference Manual for the MPF-III Personal Computer. In this book, we present to the readers an overall description about the integral components of the MPF-III.

There are five chapters along with a section of appendices in this manual as listed below:

Chapter 1 Getting to Know Your MPF-III Personal Computer

Chapter 2 Monitor Program

Chapter 3 Other System Software

Chapter 4 Memory Organization

Chapter 5 Input/Output

Appendices

A. Glossary

B. The 6502 Instruction

C. Hardware Structure of RS-232 Interface

D. ASCII Character Codes

E. A Summary of Built-in Subroutines

For information on the installation of the MPF-III and programming in the BASIC language, please refer to The MPF-III Operation Manual and The MPF-III BASIC Programming Manual respectively.

TABLE of CONTENTS

Chapter 1 Getting to Know Your MPF-III Personal Computer 1

- 1.1 Presenting the MPF-III Personal Computer 1
- 1.2 An Overview of the Specifications 2
- 1.3 The MPF-III Hardware Structure 5
 - 1.3.1 Main PC Board 5
 - 1.3.2 Power Supply 7
 - 1.3.3 Keyboard 8
- 1.4 Video Display 8
- 1.5 I/O Expansion Options and Interface Devices 10
 - 1.5.1 Printer Interface 10
 - 1.5.2 Cassette Interface 10
 - 1.5.3 Speaker 11

Chapter 2 Monitor Program 13

- 2.1 Entering Monitor Program. 13
 - 2.1.1 Communicating with the Monitor 13
 - 2.1.2 Reading the Contents of Memory 14
 - 2.1.3 Altering the Contents of Memory 17
- 2.2 Writing and Executing Machine Language Programs 23
- 2.3 Examining and Changing the Contents of Registers 25
- 2.4 Other Monitor Commands 26

2.5 Multiple Command Lines	28
2.6 Some Interesting Features of the Monitor	28
2.7 Summary of Monitor Commands	30
2.7.1 Examining the Contents of Memory	30
2.7.2 Altering the Contents of Memory	31
2.7.3 MOVE and VERIFY	31
2.7.4 WRITE to and READ from Tape	31
2.7.5 GO and LIST	31
2.7.6 Examining/Changing the Contents of Registers	32
2.7.7 Other Commands	32

Chapter 3 Other System Software 33

3.1 Sound Generator	33
3.1.1 Structure of MPF-III SG	33
3.1.2 Programmable Sound Generator	35
3.1.3 Software and Application	44
3.1.4 Composing Music with the Sound Generator	48

Chapter 4 Memory Organization 55

4.1 Main Memory Map	55
4.1.1 RAM Memory Allocation	56
4.1.2 The 6502 Stack	60
4.1.3 The Input Buffer	60
4.1.4 Link-address storage	61
4.1.5 The Display Buffers	61
4.2 Bank-switched Memory	62
4.2.1 Setting Bank Switches	62
4.3 Reset Routine	66
4.3.1 The Cold-start Procedure	67
4.3.2 The Warm-start Procedure	67
4.3.3 Forced Cold-start	67
4.3.4 The Reset Vector	68
4.4 Peripheral Card Memory Space	69

4.4.1 Peripheral Card I/O Space	69
4.4.2 Peripheral Card ROM Space	69
4.4.3 Expansion ROM Space	71
4.4.4 Peripheral Card RAM Space	72

Chapter 5 Input/Output 81

5.1 I/O Devices 81

5.1.1 Keyboard	81
5.1.2 Video Display Generator	86

5.2 Secondary Inputs and Outputs 96

5.2.1 Speaker	96
5.2.2 Cassette I/O	96
5.2.3 Hand Control Connector Signals	97
5.2.4 Annunciator Output	97
5.2.5 Strobe Output	98
5.2.6 Switch Input	98
5.2.7 Analog Input	99
5.2.8 Printer Interface	99
5.2.9 Sound Generator	100

5.3 Peripheral Connectors 101

5.3.1 Printer Connector	101
5.3.2 Disk Drive Connector	101
5.3.3 RS-232 Interface Connector	102
5.3.4 Keyboard Connector	102
5.3.5 Joystick Connector	103

5.4 Built-in I/O Firmware 104

5.4.1 Standard Output Features	105
5.4.2 Standard Input Features	109

Appendices

A. Glossary	115
B. The 6502 Instruction	131
C. Hardware Structure of RS-232 Interface	133

D. ASCII Character Codes	135
E. A Summary of Built-in Subroutines	139

Chapter 1

Getting to Know Your MPF-III Personal Computer

1.1 Presenting the MPF-III Personal Computer

The MPF-III Personal Computer, an improved version of the MPF-II, is a versatile microcomputer system featuring sophisticated software and hardware capabilities. It is ideal for many dedicated purposes. In this chapter, we will show you the special features of the system itself and the peripheral devices and accessories.

As shown in Fig. 1-1, the system unit of the MPF-III looks quite different from MPF-II. It takes the shape of a rectangular box. Two disk drives are placed in parallel which can be piled on the system unit as shown in Fig. 1-2. The options such as Z-80 CP/M card, floppy disk interface can be installed inside the system unit, which makes the MPF-III a compact, solid, easy-to-transport facility.

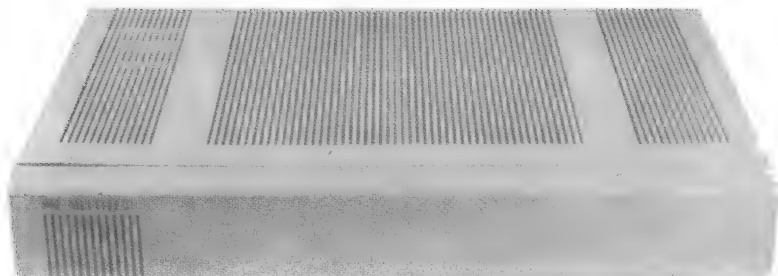


Fig. 1-1 The System Unit of MPF-III



Fig. 1-2 The MPF-III Personal Computer

1.2 An Overview of the Specifications

1) CPU:

6502 microprocessor

2) ROM

A total of 24K memory capacity, including 11K BASIC interpreter, 4.75K monitor program, 1.25K sound generating program, 3K 80-column text display program, and 1.5K printer driver.

3) RAM:

A total of 66K memory capacity, including 2K 80-column text display buffer.

4) Screen Display:

Text mode : 24 lines with 40 or 80 characters on each line can be displayed.

Low-resolution graphics mode : 40 x 48 blocks can be displayed in any of 16 colors.

High-resolution graphics mode : 280 x 192 dots can be displayed in any of 6 colors.

5) Keyboard

The MPF-III has a separate keyboard of ergonomic design. There are a total of 90 keys on the keyboard. In addition to 56 general-purpose keys, there are :

F1--F12 key (User-defined key)

PB0--PB1 key

BREAK key

RESET key

LIST key

RUN key

DELC key

INSC key

HALT key

ALT key

Numeric pad

6) Printer Interface and Driver

The MPF-III can be connected to EPSON, C.Itoh, or CP-80 printers without any modification of the software.

7) Cassette recorder can be connected to the MPF-III as an external source for data storage or retrieval.

8) Display Interface

Either a TV set or video monitor can be used for screen display.

9) Joystick Interface

The joystick may be used.

10) Sound Generator

With BASIC special sound effect commands you can have the sounds of the machine gun, dropping a bomb, explosion, and the laser gun. The sound of the piano, the organ, the bell and the xylophone can be played in the range of three octaves with rhythm accompaniments of waltz, tango, rumba, disco, swing, blues, rock, and chacha. The duration of each sound is in the range from that of a whole note to that of a sixty-fourth note.

11) Disk Drive Interface

Disk drives can be connected to the MPF-III.

12) Z-80 CP/M card can be added to run CP/M and Microsoft BASIC on the MPF-III.

To the right of the 6502 CPU you can find four slots numbered P2, P4, P6, P7 respectively. As a rule, floppy disk interface is inserted in P6, Z-80 CP/M card is inserted in P7, CCC (Chinese Character Card) is inserted in P4, P2 is for external expansion. To the left of slot P4 you can find three IC numbered 2764, these combine to constitute the 24K ROM. The ROM includes:

1. Printer interface driver
2. Sound generator
3. 80-column software
4. MPF-III cassette interface software
5. Screen editor
6. BASIC interpreters
7. System monitor

To the left of ROM, you can find an IC numbered 8912 which is a Sound Generator.

On the left of the main PC board you can find a row of eight ICs numbered 4164 together with a 6116 to the right. They are the RAM of MPF-III, which include 64K dynamic RAM and 2K static RAM for 80-column text display.

These RAM serve as the storage space for your programs and data when you run programs. All the programs and data stored in RAM will disappear if the power supply is turned off.

On the top of the main PC board you can find a volume regulator marked VR1 which is used to adjust the volume of sounds.

To the right of the volumn regulator you can find a row of four miniature switches, the functions of these switches are described below. (You can press them down to get the ON positions.)

- S1: on (See note below)
off 40-column text display
- S2: on Connection to the EPSON or CP-80 printer
off Connection to the C.Itoh printer
- S3: on Channel 3 (NTSC only)
off Channel 4 (NTSC only)
- S4: on 64K RAM is selected
off 48K RAM is selected

Note: The state of S1 is reserved for other purposes, before you power on the MPF-III, be sure to set this switch to the off position. To set the MPF-III in the 80-column text mode, use the command PR#3.

1.3.2 Power Supply

A switching power supply is built in MPF-III. The input voltage is 110 V AC and the output voltages are +5V +12V -5V and -12V. Below is the pin configuration of the power cord connector. (Fig. 1-4)

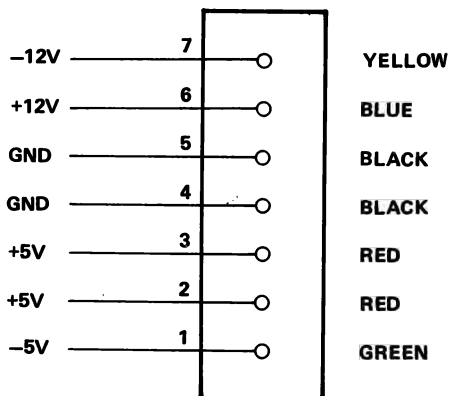


Fig. 1-4 Power Cord Connector

1.3.3 Keyboard

The MPF-III keyboard consists of 90 keys and two LED indicators indicating status of CAPS LOCK and NUM LOCK. The keyboard is connected to the system unit by a cable and a nine-pin D-type connector. The keyboard is of ergonomic design, smart in appearance and easy to operate.

Below is a list of the pin assignments of the MPF-III keyboard cable.

PIN	signal
1	PB1
2	+5V
3	GND
4	DATA
5	CLOCK
6	STROBE
7	PB0
8	RESET
9	AKD

PB0, PB1 : Signals read into the system unit. AKD stands for "any key down". If any key is pressed, the signal is high; otherwise it is low.

1.4 Video Display

On the back panel of the MPF-III system unit, you will find two connectors marked MONITOR and TV respectively. You need an RF modulator if you use a TV set instead of a monitor. The TV set must be adjusted to a specified channel.

There are three display modes available on MPF-III. They are Text mode, Low-resolution graphics mode and High-resolution graphics mode.

In text/low-resolution graphics mode and high-resolution graphics mode, memory locations \$0400-\$0BFF (\$ stands for hexadecimal) and \$2000-\$5FFF are used as video display buffers respectively.

For either mode the source of information needed to form the screen display are in fact stored in two memory areas of the same size. One is called "primary page" and the other "secondary page". The secondary page is useful when you want an instant display. See the following table:

Mode	Page	Starting Address		Ending Address	
		Hexadecimal	Decimal	Hexadecimal	Decimal
High-resolution	Primary	\$2000	8192	\$3FFF	16383
Graphics	Secondary	\$4000	16384	\$5FFF	24575
Text/Low-Resolution	Primary	\$0400	1024	\$07FF	2047
Graphics	Secondary	\$0800	2048	\$0BFF	3071

Fig. 1-5 Screen Display Memory Locations

To select which mode and page is to be used for screen display, we use screen "switches". These switches are controlled by the software (i.e., programs) of the MPF-III. Each switch corresponds to a specific memory location. When a program is used to turn on or turn off (toggle) a switch, it simply references the specific memory location of that switch. The contents of data read from or written to the memory location does not matter at all.

The MPF-III has eight memory locations as soft switches for screen display. They come in pairs. In other words, when one of the pair is on, the other is off. The following table lists the addresses of the switches.

Hex.	Decimal	Description
\$C050	49232	Display a graphics mode
\$C051	49233	Display text mode
\$C052	49234	Display all text or graphics
\$C053	49235	Mix text and graphics mode
\$C054	49236	Display the primary page
\$C055	49237	Display the secondary page
\$C056	49238	Display low-resolution graphics mode
\$C057	49239	Display high-resolution graphics mode

Fig. 1-6 Screen Soft Switches Location

1.5 I/O Expansion Options and Interface Devices

1.5.1 printer Interface

On the back panel of MPF-III System Unit, you can see a printer connector marked "PRINTER". The MPF-III may be connected to EPSON, CP-80 or C.ITOH printers or any other printers with parallel interface which can work well with MPF-III through this connector. The pin configuration of the printer connector is shown in Fig. 1-7.

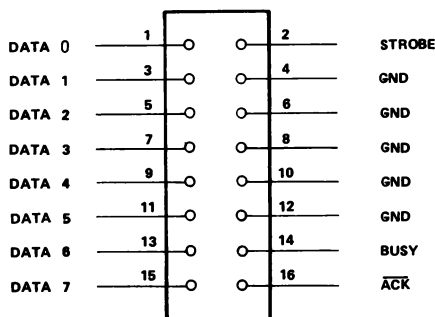


Fig. 1-7 Printer Connector

1.5.2 Cassette Interface

On the left of the back panel (When holding an MPF-III with the back panel toward you) of the MPF-III, there are two small holes marked EAR and MIC, respectively. You can plug a cable with a phone plug on each end to your MPF-III and cassette recorder. Then you can store your program from MPF-III to the tape, or load your program from the tape to the MPF-III.

Note when you load a program from the tape to the MPF-III, one end of the cable should plug to the EAR jack of the MPF-III, while the other end of the cable should be connected to the jack marked ERA or OUT on the cassette recorder. If you store a program from the MPF-III to the tape, one end of the cable should plug to the MIC jack of the MPF-III, while the other end of the cable should be connected to the jack marked MIC or IN on the cassette recorder.

The connector marked MIC is connected to another soft switch on the MPF-III main board. Like that for the speaker, the soft switch for cassette interface is also a toggle type switch (a two-position switch). The memory location associated with the switch is: 49184 (\$C020 hexadecimal). The program that converts data into recordable tone resides in the monitor program.

The other connector marked EAR is for reading program from tape to the MPF-III. Its major purpose is to decode information stored on tape and to load the decoded data into the MPF-III. Thus, a program stored on tape can be loaded back to the MPF-III for execution.

The input circuit of the MPF-III takes 1V peak-to-peak signal from the EARPHONE jack on the recorder and converts the signal to a string of ones and zeroes. Each time the signal received by the input circuit changes from positive to negative or from negative to positive, the state of the input circuit will be changed. If the input circuit was sending ones, it will start sending zeroes, and vice versa. A program can judge by the value of the memory location 49248 (\$C060) the state of the input circuit. If the value of the location is greater than or equal to 128, then the state of the input circuit is one. Otherwise, the state of the input circuit is zero.

1.5.3 Speaker

On the left of the MPF-III main board, there is a built-in speaker. You can program the speaker to generate various sounds.

The speaker is controlled by a "soft" switch. The switch can put the paper cone in two positions: "in" and "out". Every time a program references the memory location corresponding to the switch, the state of the speaker is changed. Each time the state of the speaker is changed, the speaker generates a tone. By changing the state of the speaker frequently and continuously, you can generate a tone from the speaker.

The memory location associated with the soft switch is 49200 (decimal). The hexadecimal equivalent of that value is \$C030. When this location is referenced, the speaker will generate a tone.

A program may reference this location by reading from or writing into the location. It is the "referencing" of the location that change the state of the speaker. The value that is read from or written into the memory location has nothing to do with the flipping of the soft switch for speaker.

Note that when the 6502 microprocessor performs a "write" operation, it must first perform a "read" operation. Therefore, if you reference the soft switch by writing a value into its associated location, you are actually throwing the switch twice. To a toggle-type switch like the soft switch for the speaker, after a "write" operation is completed, the state of the switch remains unchanged.

Chapter 2

Monitor Program

2.1 Entering Monitor Program

The entry point of MPF-III monitor program is at the memory location \$FF69, it is 65385 or -151. You can enter the monitor by calling this location, using the CALL-151 instruction.

Once you entered the monitor program, the monitor prompt * and cursor will appear at the left edge of the screen. Now the monitor is ready to accept a standard input line. After you have typed an input line, the monitor will not respond unless you press the carriage return key which is marked . Each input line to the monitor may consist of up to 255 characters. If you want to exit the monitor, you can press the CTRL, C, and carriage return or type 3D0G. Another way to do this is to press the RESET-CTRL key.

2.1.1 Communicating with the Monitor

To communicate with the monitor, you must give three types of information to it: address, data, and command. Data and addresses should be given in hexadecimal. The decimal numerals from 0 through 9 are used to represent hexadecimal numerals from 0 through 9, and the letters A, B, C, D, E, F, are used to represent the decimal numbers from 10 to 15. Thus, a hexadecimal can be used to represent any of the 16 decimal numbers from 0 to 15, and two hex digits can be used to represent any of the 256 decimal numbers from 0 to 255, and four hex digits can be used to represent 65536 decimal numbers from 0 to 65535. In the MPF-III, every memory location is numbered with four hex digits, and the contents of each memory location are represented by two hex digits.

When the monitor looks for an address, it is actually looking for a group of four hex digits. If a hex digit group (an address) has less than four hex digits, the monitor assumes the hex digit group comes with leading zero(s). If an address contains more than four hex digits, the monitor will truncate the number and only receive the last four hex digits. The monitor handles the data in a similar way.

The monitor recognizes 17 different command characters. Some of them are punctuation marks, and the rest are upper-case letters and control characters. Since the screen does not echo the control characters, you must use the control characters carefully. The knowledge of two special memory locations -- the last opened location and the next changeable location -- is necessary in communication with the monitor. The last opened location is the location you last accessed. The next changeable location is the location you may access immediately. Frequently, the last opened location is identical with the next changeable location. You would learn more about the two locations later in this book.

2.1.2 Reading the Contents of Memory

1) Reading the contents of a memory location

Type in an address on the input line, the monitor will respond by displaying the address you just typed and the contents of this address. For example,

```
*F800
```

```
F800-4C
```

```
*F900
```

```
F900- F8
```

```
*
```

Every time the monitor displays the contents of an address, it remembers that location as the last opened location. It also regards the location as the next changeable location.

2) Reading the contents of a memory block

Typing a period (.) and an address will cause the monitor to perform a memory dump, dumping the contents from the last opened location to the location you typed following the period.

```
*FA00.  
FA00- D8  
*.FA0F  
  
FA01- 62 5A 48 26 62 94 88  
FA08- 54 44 C8 54 68 44 E8 94  
*F800  
  
F800- 4C  
*.F812  
  
F801- CA EF 4A 08 20 F5 FB  
F808- 20 73 FC A5 09 28 90 02  
F810- 69 0F 85  
*.F82A  
  
F813- 09 60 86 EA EA  
F818- EA 20 00 FB C4 2C B0 11  
F820- C8 20 C8 EF 90 F6 69 01  
F828- 48 20 00  
*
```

While performing a memory dump, note the following:

- a. The memory dump begins with the contents of the location immediately following the last opened location.
- b. Every output line displays contents of no more than eight locations.
- c. Except for the first output line, all other output lines start at an address with the last digit being a 0 or an 8.

When the monitor performs a memory dump, it first displays the contents of the location immediately following the last opened location. Then it displays the contents of the following locations one after the other until it proceeds to the location whose address ends with 0 or 8. And then the contents of the following locations are displayed in a new output line. The monitor will continue dumping contents of memory until proceeding to the location you specified after the period. When the monitor stops memory dumping, the location you specified will be considered by the MPF-III as the last opened address.

If the address you specified is less than the address of the last opened location, then only the contents of the single location immediately following the last opened location are to be displayed.

A simpler way to look into the contents of a memory range (block) is to type in the address of the last opened location, a period, and the ending address.

*F200.F22F

```
F200- 00 1C 22 2A 3A 1A 02 3C
F208- 00 08 14 22 22 3E 22 22
F210- 00 1E 22 22 1E 22 22 1E
F218- 00 1C 22 02 02 02 22 1E
F220- 00 1E 22 22 22 22 22 1E
F228- 00 3E 02 02 1E 02 02 3E
*F900.F910
```

```
F900- F8 69 BF 20 ED FD CA D0
F908- EC 20 48 F9 A4 2F A2 06
F910- E0
*
```

- 3) Dumping memory by pressing the carriage return key:
Another simple way to perform a memory dump

By pressing the carriage return key, the MPF-III will perform a memory dump to display an output line beginning with the contents of the location immediately following the last opened location. At the same time, the monitor will remember the address of the last displayed location as the last opened location.

*F805

F805- 20

* 

F5 FB

* 

F808- 20 73 FC A5 09 28 90 02

*F832

F832- A0

* 

2F D0 02 A0 27

* 

F838- 84 2D A0 27 A9 00 85 30

*

2.1.3 Altering the Contents of Memory

1) Altering the contents of a single memory location

We have mentioned before that the last opened location is usually the next changeable location. To change the contents of a location, type ":" and the data you intend to write into the location.

*4020

4020- E9

*:2E

*4020

4020- 2E

Now the value of the next changeable location has been changed to 2E. If you are not certain of the last opened location, simply type the address of the location whose contents you intend to change, a colon, and the data you want to write to that location. After you have typed in the new data, the original data was changed.

*4020:3F

*4020

4020- 3F

*

2) Altering the contents in consecutive memory locations

To change the contents in consecutive memory locations, you don't have to type an address, a colon, and a value for each location. The monitor allows you to change the contents of up to 85 consecutive memory locations at a time. All you have to do is to type in the starting address of the memory range, a colon, and a series of data separated by spaces. If you omit the starting address, the MPF-III assumes that the next changeable location is the starting address.

```
*4000:12 23 34 45 56 67 78 89
```

```
*4000
```

```
4000- 12
```

```
* 
```

```
23 34 45 56 67 78 89
```

```
*4020:0 1 2 3
```

```
*:4 5 6 7
```

```
*4020.4027
```

```
4020-00 01 02 03 04 05 06 07
```

```
*
```

3) Moving a memory range to a new location

The MOVE command allows you to move a whole memory range to a new location. To complete this task, you have to tell the monitor:

- a. The destination where you wish to move the memory range to.
- b. The starting address of the memory range.
- c. The last address of the memory range.

The first letter of the MOVE command "M" should follow the memory range in upper case. The "<" mark is used to point to the destination. To move a memory range, you should first tell the monitor the destination by typing the address of the destination, followed by the starting address of the memory location, a period, the last address, and finally the upper case command M. The format of the command looks like

[Destination] < [Starting address] [Ending address]

*4060.406F

4060- 00 00 00 00 00 00 00 00

4068- 00 00 00 00 00 00 00 00

*4000:00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF

*4000.400F

4000- 00 11 22 33 44 55 66 77

4008- 88 99 AA BB CC DD EE FF

*4060<4000.400FM

*4060.406F

4060- 00 11 22 33 44 55 66 77

4068- 88 99 AA BB CC DD EE FF

*4070.4077

4070- 00 00 00 00 00 00 00 00

*4070<4000.4007M

*4070.4077

4070- 00 11 22 33 44 55 66 77

*

If the value of the last address is less than the value of the starting address, then only the contents of one memory location will be moved to the destination. If the address of destination is inside the memory range to be moved, then the following may happen:

*4020.4037

4020- 23 01 D0 F6 60 FF 00 01

4028- 02 03 04 05 00 01 02 03

4030- 04 05 00 01 02 03 00 00

*4020:00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF

*4026<4020.402FM

*4020.4037

4020- 00 11 22 33 44 55 00 11

4028- 22 33 44 55 00 11 22 33

4030- 44 55 00 11 22 33 00 00

*

4) Comparing the Contents of Two Memory Ranges

You can use the monitor to compare the contents of two memory ranges, using the same command format as you use to move a memory range. The command format is listed as follows:

[Destination] < [Starting address] [Ending address]V

where the upper case letter "V" is the first letter of the monitor command VERIFY.

[Destination]: the starting address of a memory range whose contents are to be compared with those of another memory range.

[Starting address] is the starting address of the memory range you want to compare its contents with those contained in the destination memory range;

[Ending address] is the ending address of the memory range you want to compare its contents with those contained in the destination memory range.

See the following example:

*4000:00 11 22 33 44 55 66 77

*4010<4000.4007M

*4010<4000.4007V

*4006:AA

*4010<4000.4007V

4006-AA(66)

*

In case the monitor finds discrepancy, it will display the address where discrepant value was found and the disagreeing values. If there is no discrepancy, nothing will be displayed. Note when discrepancy happens, the displayed address is an address in the original memory range specified by [starting address] and [Ending address].

5) Saving the Contents of a Memory Range on Tape

The monitor command WRITE is used to save the contents of up to 65536 memory locations on cassette tape. You have to tell the monitor the beginning and last addresses of a memory range. The command formats are shown below

[Starting address]. [Ending address]W

The capital letter 'W' is the first letter of the word WRITE.

In order to allow a recorder to save data correctly, you have to set it to record mode before typing the carriage return key. Let the tape recorder run for a few seconds before pressing the carriage return key. The monitor will first write a "leading tone" unto the tape, and then write data to the tape. It will take about ten seconds for the MPF-III monitor to write the leading tone into the tape.

After the monitor has saved the data of a memory range on a tape, it will generate a "beep" and display another prompt. It is a good practice to rewind the tape and label the starting and ending addresses of the memory range you just saved on the tape.

*4000:0 1 2 3 4 5 6 7 8 9 A B C D E F

*4000.400F

4000- 00 01 02 03 04 05 06 07

4008- 08 09 0A 0B 0C 0D 0E 0F

*4000.400FW

WAIT OK

*

In addition to the leading tone, it takes 35 seconds to write the contents of 4,096 memory locations to a tape. The monitor will write a checksum on the tape after it has written all the contents of the memory range. The checksum is a partial sum of all the values of the memory range. When the MPF-III reads back the contents of the memory range from the tape in the future, it will determine if the reading is performed without mistake by examining the checksum.

6) Reading back a memory range from tape

You can use the READ command to read a memory range back into the MPF-III from the tape on which you had saved the memory range with the WRITE command.

The data saved on the tape is not required to be read back to the memory range in the MPF-III where the data previously occupied. It suffices if you put the contents of a memory range to any RAM area in the MPF-III, provided the size is exactly the same as the memory range.

The format of the READ command is exactly the same as that of the WRITE command except that W is replaced by R as shown below:

[Starting address]. [Ending address]R

Remember that after you have typed in monitor commands, don't press the carriage return key too hastily. you have to press the PLAY button on the recorder first and press  the moment you hear the leader tone. Because the MPF-III needs about three seconds to lock onto the frequency, you shall wait for several seconds to let the tape pass by before pressing the carriage return key.

```
*4000:0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
*4000.400F
```

```
4000- 00 00 00 00 00 00 00 00
```

```
4008- 00 00 00 00 00 00 00 00
```

```
*4000.400FR
```

```
WAIT OK
```

```
*4000.400F
```

```
4000- 00 01 02 03 04 05 06 07
```

```
4008- 08 09 0A 0B 0C 0D 0E 0F
```

*

After the monitor has finished reading data from tape, it will produce a checksum. The monitor will compare this checksum with that it just read in from tape. If the two match, it will produce a prompt on the screen. If the two disagree, the ERR message will display on the screen.

2.2 Writing and Executing Machine Language Programs

Though it is rather difficult to write a machine language program, machine language is still the most efficient language on the MPF-III. The monitor of the MPF-III offers several features, enabling you to write and execute your machine language program.

How to write a machine language program? It's easy. You can use the monitor commands we have just discussed to enter the hexadecimal equivalents of the operation codes (opcode) and operands of a machine language program into the RAM of the MPF-III.

You can also use the monitor commands to change, move, compare the machine language programs within the RAM of the MPF-III. Even more, you can use the monitor command to store a machine language program onto tape or read a program from tape. The most important command to machine language programs is the GO command. Typing in the starting address of a machine language program and the capital letter "G" will make the monitor execute the program. The monitor program considers that program as one of its subroutine. So the last instruction of that program must be RTS (return from subroutine), enabling control to be transferred to the monitor.

Your machine language program can call many useful subroutines contained in the monitor program to perform the tasks you desire. The following is an example program which calls a subroutine to display the 26 letters of the alphabet on the screen.

```
*4000:A9 C1 20 ED FD 18 69 01 C9 DB D0 F6 60 FF FF FF
```

```
*4000.400F
```

```
4000-A9 C1 20 ED FD 18 69 01
```

```
4008-C9 DB D0 F6 60 FF FF FF
```

```
*4000G
```

```
ABCDEFGHIJKLMNPOQRSTUVWXYZ
```

```
*
```

It will be very difficult to read and debug a program written in hexadecimal. Thus, a LIST command is designed to list machine language program in the easy-to-read assembly language form. Each time the LIST command is given to the monitor, the MPF-III will display 20 lines of instruction. The mnemonic is displayed on the right side on each line, and the opcode, operand, and the hexadecimal numbers are displayed to the left of the mnemonic.

*4000L

4000-	A9 C1	LDA #C1
4002-	20 ED FD	JSR \$FDED
4005-	18	CLC
4006-	69 01	ADC #\$01
4008-	C9 DB	CMP #\$DB
400A-	D0 F6	BNE \$4002
400C-	60	RTS
400D-	FF	???
400E-	FF	???
400F-	FF	???
4010-	00	BRK
4011-	00	BRK
4012-	00	BRK
4013-	00	BRK
4014-	00	BRK
4015-	00	BRK
4016-	00	BRK
4017-	00	BRK
4018-	00	BRK
4019-	00	BRK

*

The lines above are the program we discussed a moment ago. The lines with ??? is just there to fill up the screen.

2.3 Examining and Changing the Contents of Registers

Even though the actual contents of the 6502's internal registers are changing as you use the Monitor, you can examine the values that the registers contained at the time the Monitor gained control, either because you called it or because the program you are debugging stopped at a break (BRK). You can also store new register values that will be used when you execute a program from the Monitor using the GO command, described below.

When you call the Monitor, it stores the contents of the 6502 registers in memory. The registers are stored in the order A,X,Y,P(processor status register), and S (stack pointer), starting at location \$45 (decimal 69). When you give the Monitor a GO command, the Monitor loads the registers from these five locations before it executes the first instruction in your program.

Monitor Register Commands

Typing CTRL-E and pressing  invokes the Monitor's EXAMINE command, which displays the stored register values and sets the location containing the contents of the A-register as the next changeable location. After using the EXAMINE command, you can change the values in these locations by typing a colon and then typing the new values separated by spaces. In the following example, you display the registers, change the first two, and then display them again to verify the change.

```
* CTRL-E
```

```
A=0A X=FF Y=D8 P=B0 S=F8
```

```
*:B0 02
```

```
* CTRL-E
```

```
A=B0 X=02 Y=D8 P=B0 S=F8
```

2.4 Other Monitor Commands

You can set the contents of the memory location used by the CONOUT subroutine to switch the video display to either Inverse or Normal mode. The monitor commands NORMAL and INVERSE are used to set the video display mode.

*F800.F80F

F800-20 D6 F8 A9 F0 B0 02 49

F808-FF 85 2E D0 01 C8 B1 26

*I

*F800.F80F

F800-20 D6 F8 A9 F0 B0 02 49

F808-FF 85 2E D0 01 C8 B1 26

*N

*F800.F80F

F800-20 D6 F8 A9 F0 B0 02 49

F808-FF 85 2E D0 01 C8 B1 26

*

Type in the preceding program and see how it works.

* Back to BASIC

Use the BASIC command, CTRL-B, to leave the Monitor and enter the BASIC that was active when you entered the Monitor. Any program or variables that you had previously in BASIC will be lost. If you want to re-enter BASIC with your previous program and variables intact, use CTRL-C. If you are using the MPF-DOS, press CTRL-RESET or type

3D0G

to return to the language you were using, with your program and variables intact.

* Redirecting Input and Output

The PRINTER command, activated by a CTRL-P, diverts all output normally destined for the screen to an interface card in a specified expansion slot, from 1 to 7. There must be an interface card in the specified slot, or you will lose control of the computer and your program and variables may be lost. The format of the command is

{slot number} CTRL-P

A PRINTER command to slot number 0 will switch the stream of output characters back to the video display.

In much the same way that the PRINTER command switches the output stream, the KEYBOARD command substitutes the interface card in a specified expansion slot for the MPF-III'S normal input device, the keyboard. The format for the KEYBOARD command is:

{slot number} CTRL-K

A slot number of 0 for the KEYBOARD command directs the Monitor to accept input from the MPF-III keyboard.

The PRINTER and KEYBOARD commands are the exact equivalents of the BASIC commands PR# and IN#.

The system monitor can also perform some simple hexadecimal addition and subtraction. You can do some examples by following the procedures listed below.

[Value] + [Value]

[Value] - [Value]

[Value] represents a hexadecimal number. See the following examples;

```
*10+12
=22
*3A-B
=2F
*EE+3
=F1
*5-8
=FD
*
```

2.5 Multiple Command Lines

You can put as many Monitor commands on a single line as you like, as long as you separate them with spaces and the total number of characters in the line is less than 254. Adjacent single-letter commands such as L, S, I, and N need not be separated by spaces.

You can freely intermix all of the commands except the STORE (:) command. Since the Monitor takes all values following a colon and places them in consecutive memory locations, the last value in a STORE must be followed by a letter command before another address is encountered. You can use the NORMAL command as the required letter command in such cases; it usually has no effect and can be used anywhere.

In the following example, you display a range of memory, change it, and display it again, all with one line of commands.

```
*4000.4008 4000:01 02 03 04 N 4000.4003
```

```
4000- A9 C1 20 ED FD 18 69 01
```

```
4008- C9
```

```
4000- 01 02 03 04
```

If the Monitor encounters a character in the input line that it does not recognize as either a hexadecimal digit or a valid command character, it executes all the commands on the input line up to that character, then comes to a halt.

2.6 Some Interesting Features of the Monitor

We have mentioned that the MOVE command can be used to copy the data in a memory range into another range of memory. It can fill up a memory range with the data we set. Therefore, we can store a few data values into a memory range.

```
*4000: 11 22 33 44
```

By this we have entered four data values into a memory range. Then let us give a special command to the monitor

```
[ADDR 1+NUMBER]<[ADDR 1].[ADDR 2-NUMBER]M
```

In the preceding line NUMBER represents the number of values. In this example, the number of values is four. The special MOVE command you just entered will duplicate the four values as a set pattern and fill the pattern throughout the memory range specified by ADDR 1 and ADDR 2.

```
*4000:11 22 33 44
```

```
*4004<4000.401BM
```

```
*4004.401F
```

```
4000- 11 22 33 44 11 22 33 44
```

```
4008- 11 22 33 44 11 22 33 44
```

```
4010- 11 22 33 44 11 22 33 44
```

```
4018- 11 22 33 44 11 22 33 44
```

```
*
```

You can play the same trick with the VERIFY command to check whether a pattern of values has been stored in a memory range or if the locations of a memory range contain the same value.

```
*4000:0
```

```
*4001<4000.401EM
```

```
*4001<4000.401EV
```

```
*4004:02
```

```
*4001<4000.401EV
```

```
4003-00 (02)
```

```
4004-02 (00)
```

```
*
```

* Repeating Commands

You can create a command line that repeats one or more commands over and over. You can do this by beginning the part of the command line that you want to repeat with a letter command, such as N, and ending it with the sequence 34:n, where n is a hexadecimal number that specifies the position in the line of the command where you want to start repeating; for the first character in the line, n=0. The value for n must be followed with a space in order for the loop to work properly.

The only way to stop a loop like this is to press CTRL-RESET.

```
*N 4000 4003 34:0
4000- 05
4003- 00
4000- 05
4003- 00
400
```

* Creating Your Own Commands

The USER command, CTRL-Y, forces the Monitor to jump to memory location \$3F8. You can put a JMP instruction there that jumps to your own machine-language program. Your program can then examine the Monitor's registers and pointers or the input buffer itself to obtain its data. For example, here is a program that displays everything on the input line after the CTRL-Y. The program starts at location \$300.

```
*300:A4 34 B9 00 02 20 ED FD C8 C9 8D D0 F5 4C 69 FF
```

```
*3F8:4C 00 03
```

```
* CTRL-Y THIS IS A TEST
```

```
THIS IS A TEST
```

```
*
```

2.7 Summary of Monitor Commands

2.7.1 Examining the Contents of Memory

[ADDR]

Examine the contents of a single memory location.

[ADDR1].[ADDR2]

Examine the contents of the memory range specified by ADDR 1 and ADDR 2. The  key may be used to show the contents of up to eight memory locations starting from the location immediately following the last opened location.

2.7.2 Altering the Contents of Memory

[ADDR]:[VAL 1] [VAL 2].....

Put the values behind the colon into the memory range starting at ADDR.

: [VAL 1] [VAL 2].....

Put the values following the order VAL 1, VAL 2 ... into the memory range starting at the location immediately following the last changed location.

2.7.3 MOVE and VERIFY

[ADDR 1]<[ADDR 2].[ADDR 3]M

Move the contents from ADDR 2 through ADDR 3 to the memory range with the starting address ADDR 1.

[ADDR 1]<[ADDR 2].[ADDR 3]V

Verify the contents of the memory range from ADDR 2 through ADDR 3 with the memory range starting at ADDR 1.

2.7.4 WRITE to and READ from Tape

[ADDR 1].[ADDR 2]W

Write the data values from the memory range ADDR 1 through ADDR 2 onto a tape.

[ADDR 1].[ADDR 2]R

Read data back from tape into the specified memory range and generate error message "ERR" if the reading is not successful.

2.7.5 GO and LIST

a. [ADDR]G

Transfer control to the machine language program starting at ADDR and execute the program.

b. [ADDR]L

List 20 machine language instructions beginning at ADDR.

2.7.6 Examining/Changing the Contents of Registers

CTRL-E

Examine and change the contents of 6502 CPU registers.

2.7.7 Other Commands

I Set the video display to inverse mode.

N Set the video display to normal mode.

CTRL-C

Reenter into MPF-III BASIC

[VAL 1]+[VAL 2]

Performs hexadecimal addition and print the sum.

[VAL 1]-[VAL 2]

Perform a hex subtraction and print the result.

CTRL-B

Leave the monitor and enter the BASIC that was active when you entered the monitor.

[SLOT NUM] CTRL-P

Diverts output to the device whose interface card is in slot [SLOT NUM].

[SLOT NUM] CTRL-K

Directs the monitor to accept input from the device whose interface card is in the slot designated by [SLOT NUM].

CTRL-Y

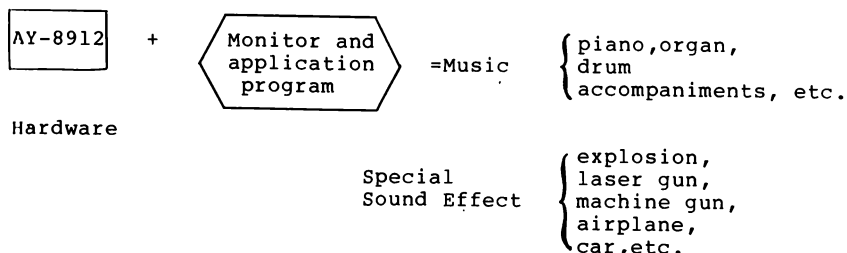
Jumps to the machine language subroutine at location \$3F8

Chapter 3

Other System Software

3.1 Sound Generator

The MPF-III SG, a combination of hardware (AY-8912) and software (the program) can produce the sounds of the piano, bell, organ and xylophone, as well as special sound effects such as the simulation of a car, a train, an airplane and the sounds of a laser gun or explosion.



3.1.1 Structure of MPF-III SG

In electronic tones, it is possible to produce the four waveforms as shown in Fig. 3-1 with a noise generator, waveform generator or filter. To human ears, the waveforms (a), (b), (c), and (d) in Fig. 3-1 will sound like buzzing sound, guitar, airplane propeller and piano respectively.

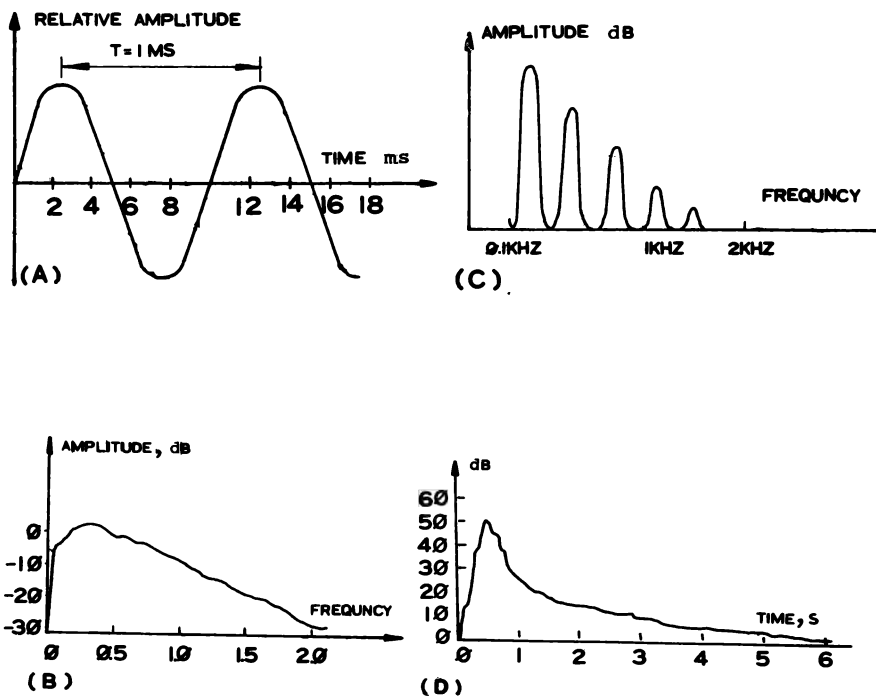


Fig. 3-1 Waveforms of Various Sounds

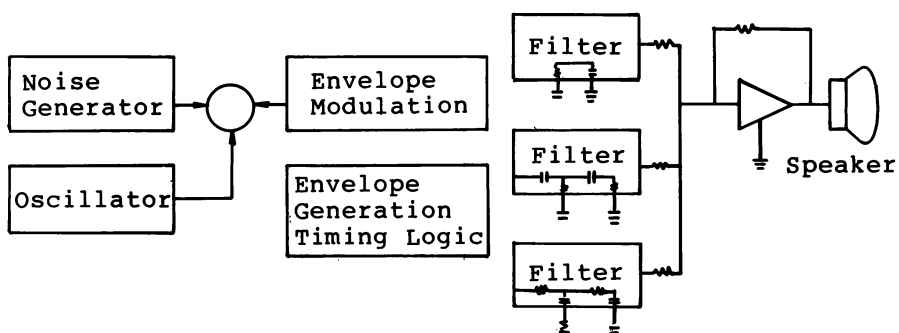


Fig. 3-2 Producing sound effects with hardware

Electronic music synthesized only with hardware can be applied in a limited number of specified fields, such as to produce only tones of electronic organ or flute.

The MPF-III SG, through the combination of software and PSG(Programmable Sound Generator) can produce better sound effect and musical composition rich in variation. By programming AY-8912, it is possible to obtain the timbre of desired musical instruments, add different accompaniment, scale the duration of musical tones and produce various special waveforms. Fig. 3-3 shows the structure of the MPF-III SG.

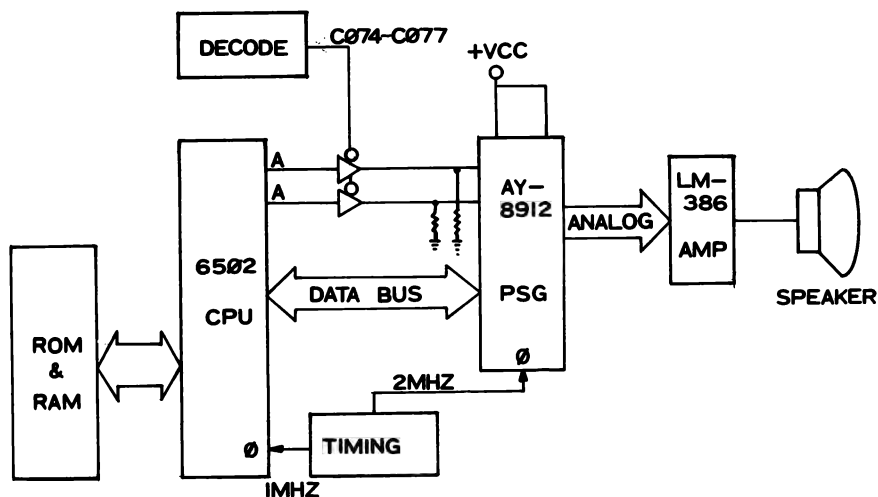


Fig. 3-3 Block Diagram of MPF-III SG

3.1.2 Programmable Sound Generator

AY-8910 and AY-8912, manufactured by GI(General Instrument) are two types of PSG (Programmable Sound Generator), similar to each other. The mere difference between them is that AY-8910 has one I/O port more than AY-8912. See Fig. 3-4 (a), (b) and (c)

Structural Components of AY-8912:

1. Noise generator
2. Tone generator
3. Amplitude controller
4. Modulation controller
5. Mixer
6. D/A converter

The six circuitries stated above are all controlled by the Register Array. All software controls are executed via these 16 registers.

The Register Array on AY-8912 is subscripted in octal. Thus, R10-R17 are R8-RF in hexadecimal. The MPF-III provides a 2-MHZ system clock for AY-8912 as an audio frequency generation reference.

Formula for Tone

$$f = f_{\text{clock}} / (16 * N1) \quad N1 = 1-4096$$

Formula for Noise

$$f = f_{\text{clock}} / (16 * N2) \quad N2 = 1-32$$

Formula for Envelope

$$f = f_{\text{clock}} / (256 * N3) \quad N3 = 1-65536$$

REGISTER		BIT		B7	B6	B5	B4	B3	B2	B1	B0
R0	CHANNEL A TONE PERIOD	8 BIT FINE TUNE A									
R1						4 BIT COARSE TUNE A					
R2	CHANNEL B TONE PERIOD	8 BIT FINE TUNE B									
R3						4 BIT COARSE TUNE B					
R4	CHANNEL C TONE PERIOD	8 BIT FINE TUNE C									
R5						4 BIT COARSE TUNE C					
R6	NOISE PERIOD					5 BIT PERIOD CONTROL					
R7	ENABLE	IN/OUT		NOISE			TONE				
		IOB	IOA	C	B	A	C	B	A		
R10	CHANNEL A AMPLITUDE					M	L3	L2	L1	L0	
R11	CHANNEL B AMPLITUDE					M	L3	L2	L1	L0	
R12	CHANNEL C AMPLITUDE					M	L3	L2	L1	L0	
R13	ENVELOPE PERIOD	8 BIT FINE TUNE E									
R14		8 BIT COARSE TUNE E									
R15	ENVELOPE/CYCLE							CONT	ATT	ALT	NOLD
R16	I/O PORT A DATA STORE	8 BIT PARALLEL I/O ON PART A									
R17	I/O PORT B DATA STORE	8 BIT PARALLEL I/O PART A									

FIG. 3-4 (b) PSG Register Array

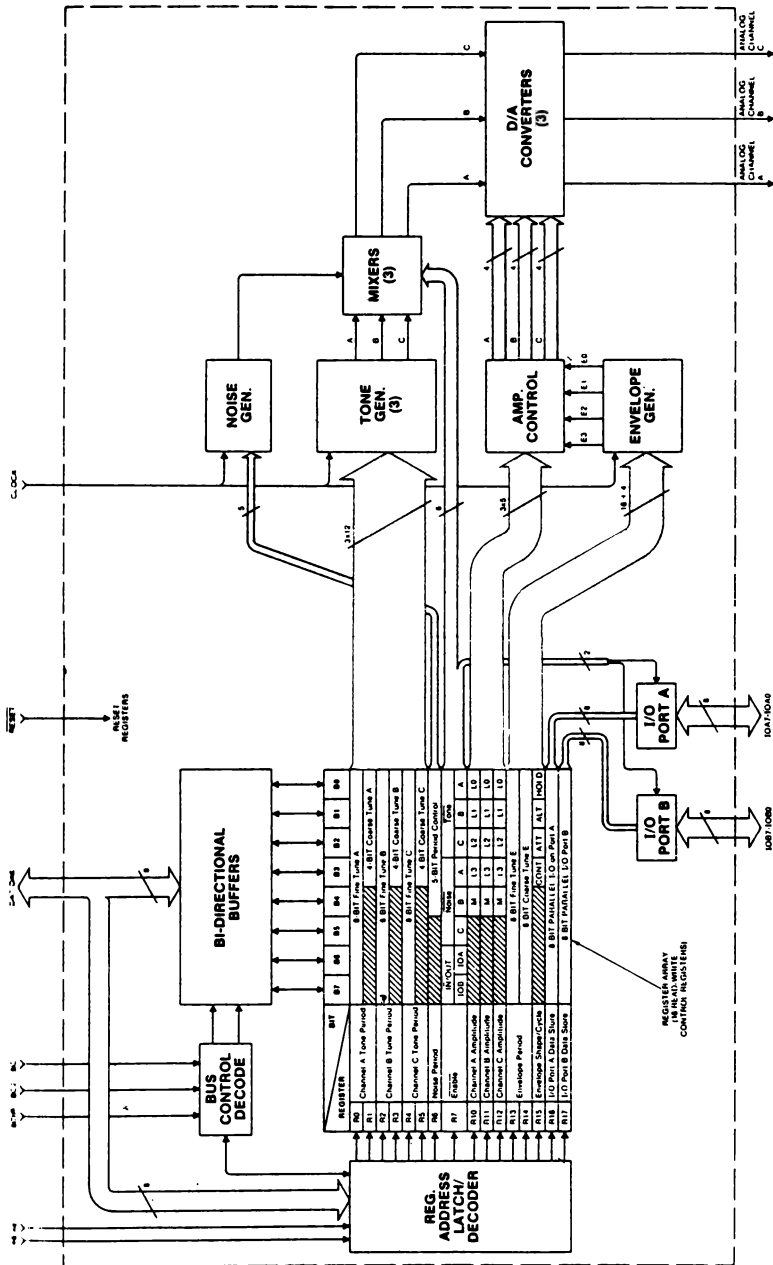


Fig. 3-4 (a) Functional Diagram of AY-8912 PSG

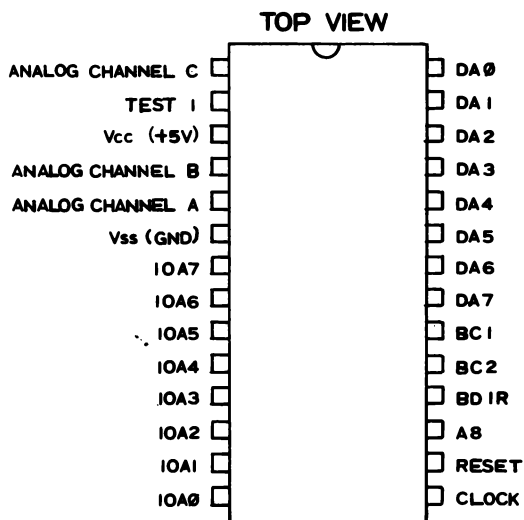


Fig. 3-4 (c) AY-3-8912 Pin Assignments

Below we give you a description on the function and usage of the 16 registers of 8912:

f: frequency requested by the user

f clock: 2MHZ

CT: Coarse Tune

FT: Fine Tune

- (1) R0-R5: These registers control a total of three tone generators, every two registers provide a binary 12-bit for controlling the frequency variation of a channel of tone. The alternatives of variation sums up to a total of $2^{12}=4096$.

See the following formula:

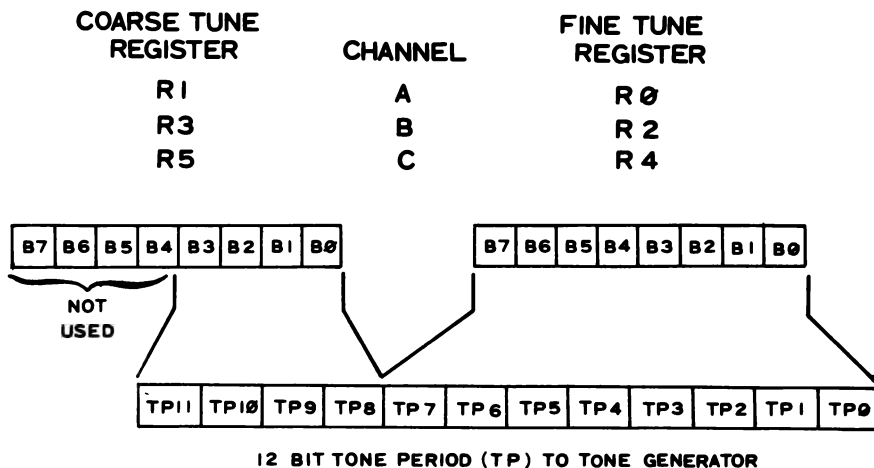
$$f = f \text{ clock} / (16 * (CT * 256 + FT))$$

On the MPF-III, the lower and upper limits of frequency is

$$f \text{ min: } 2\text{MHZ} / (16 * 4096) = 30.5\text{HZ}$$

$$f \text{ max: } 2\text{MHZ} / (16 * 1) = 125 \text{ KHZ}$$

The allocation of the registers together with a few examples are shown below:



$$(a) \quad TP_{10} = f_{\text{clock}} / 16f_T \quad (b) \quad CT_{10} + FT_{10} / 256 = TP_{10} / 256$$

Example 1: $f_T = 1\text{KHZ}$
 $f_{\text{clock}} = 2\text{MHZ}$

$$TP_{10} = 2 \times 10^6 / 16 (1 \times 10^3) = 125$$

Substituting this result into equation (b):

$$CT_{10} + FT_{10} / 256 = 125 / 256$$

$$\begin{aligned} \text{Therefore, } CT_{10} &= 0 &= 0000 (B3-B0) \\ FT_{10} &= 125 &= 01111101 (B7-B0) \end{aligned}$$

Example 2: $f_T = 100\text{HZ}$
 $f_{\text{clock}} = 2\text{MHZ}$

$$TP_{10} = 2 \times 10^6 / 16 (1 \times 10^2) = 1250$$

Substituting this result into equation (b):

$$CT_{10} + FT_{10} / 256 = 1250 / 256 = 4 + 226 / 256$$

$$\begin{aligned} \text{Therefore, } CT_{10} &= 4 &= 0100 (B3-B0) \\ FT_{10} &= 226 &= 11100010 (B7-B0) \end{aligned}$$

(2) R 6: Register 6 controls the Noise Generator. The register provides five bits, constituting a range from 1 through 32 (2^5).

Formula:

$$f = f_{\text{clock}} / (16 * FT)$$

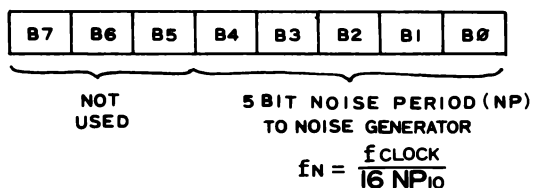
On the MPF-III, the upper and lower limits are :

$$f_{\text{max}} = 2\text{MHZ} / (16 * 1) = 125 \text{ KHZ}$$

$$f_{\text{min}} = 2\text{MHZ} / (16 * 32) = 3.9 \text{ KHZ}$$

Consult the following chart:

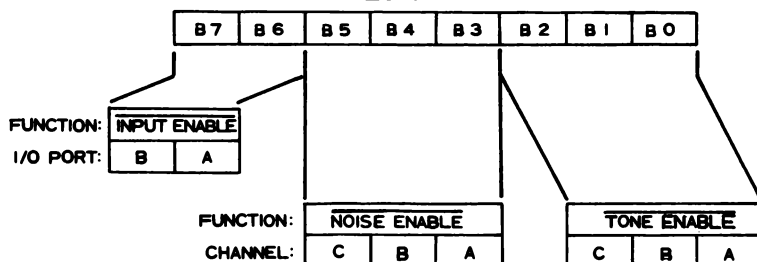
NOISE PERIOD REGISTER R6



(3) R 7: Register 7 is used to control the TONE-NOISE Mixer and enable/disable the I/O port. For each bit, the state low signifies ENABLE and the state high signifies DISABLE. Bits 0, 1, 2, take care of TONE, bits 3, 4, 5 are for NOISE, and Bits 6, 7 are used to control I/O.

Consult the following chart and tables:

MIXER CONTROL-I/O ENABLE REGISTER R7



Noise Enable Truth Table:

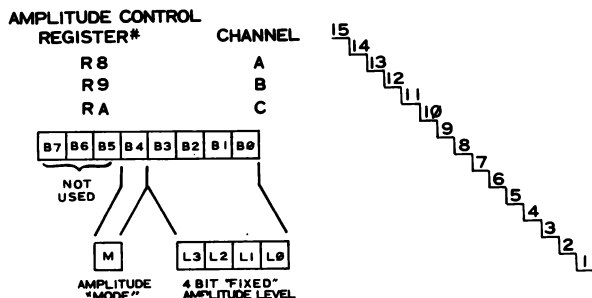
R7 Bits			Tone Enabled		
B5	B4	B3	on Channel		
0	0	0	C	B	A
0	0	1	C	B	-
0	1	0	C	-	A
0	1	1	C	-	-
1	0	0	-	B	A
1	0	1	-	B	-
1	1	0	-	-	A
1	1	1	-	-	-

Tone Enable Truth Table:

R7 Bits			Noise Enabled		
B2	B1	B0	on Channel		
0	0	0	C	B	A
0	0	1	C	B	-
0	1	0	C	-	A
0	1	1	C	-	-
1	0	0	-	B	A
1	0	1	-	B	-
1	1	0	-	-	A
1	1	1	-	-	-

- (4) R8, R9, RA: These registers are used to control the amplitude of output signal of channels A, B and C. Bit 0 through Bit 3 constitute 16 ($=2^4$) kinds of amplitude control. Bit 4=0 signifies the amplitude is specified by Bit 0 through Bit 3. Bit 4=1 signifies the amplitude is specified by the Modulation Control Registers RB and RC while Bit 0 through Bit 3 are not used.

Consult the following chart:



(5) Rb,RC: When Bit 4 of R8, R9 and RA is 1, RB, RC provides 16 bits to control the attenuation rate of the three channels. The range covers 65536 ($=2^{16}$) choices.

Formula:

$$f = f_{\text{clock}} / 256 * (256CT + FT)$$

On the MPF-III,

$$f_{\text{max}} = 2\text{MHZ} / (256 * 1) = 7.8125 \text{ KHZ}$$

$$f_{\text{min}} = 2\text{MHZ} / (256 * 65536) = 0.12\text{HZ}$$

As RB and RC are used to control the attenuation time, it is better to convert the values into time elapse(period).

$$T_{\text{min}} = 1 / f_{\text{max}} = 128 \mu\text{s}$$

$$T_{\text{max}} = 1 / f_{\text{min}} = 8.33 \text{ sec}$$

See the following chart and example:

$$(a) EP_{10} = f_{\text{clock}} / 256 f_E \quad (b) CT_{10} + FT_{10} / 256 = EP_{10} / 256$$

Example : $f_E = 0.5\text{HZ}$

$f_{\text{clock}} = 2\text{MHZ}$

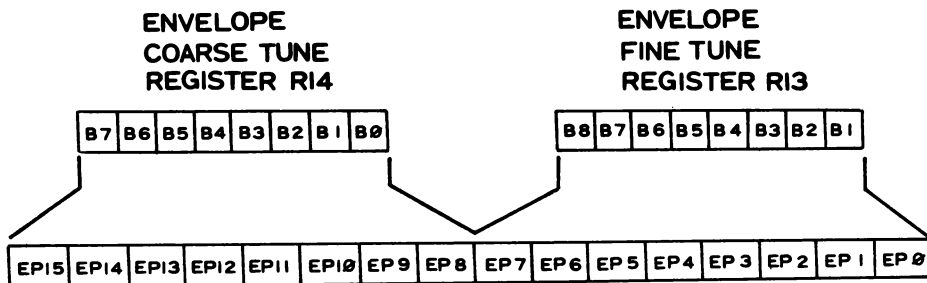
$$EP_{10} = 2 * 10^6 / 256 * 0.5 = 15.625$$

Substituting this result into equation (b):

$$CT_{10} + FT_{10} / 256 = 15.625 / 256 = 61 + 9 / 256$$

$$CT_{10} = 61 = 00111101 (B7-B0)$$

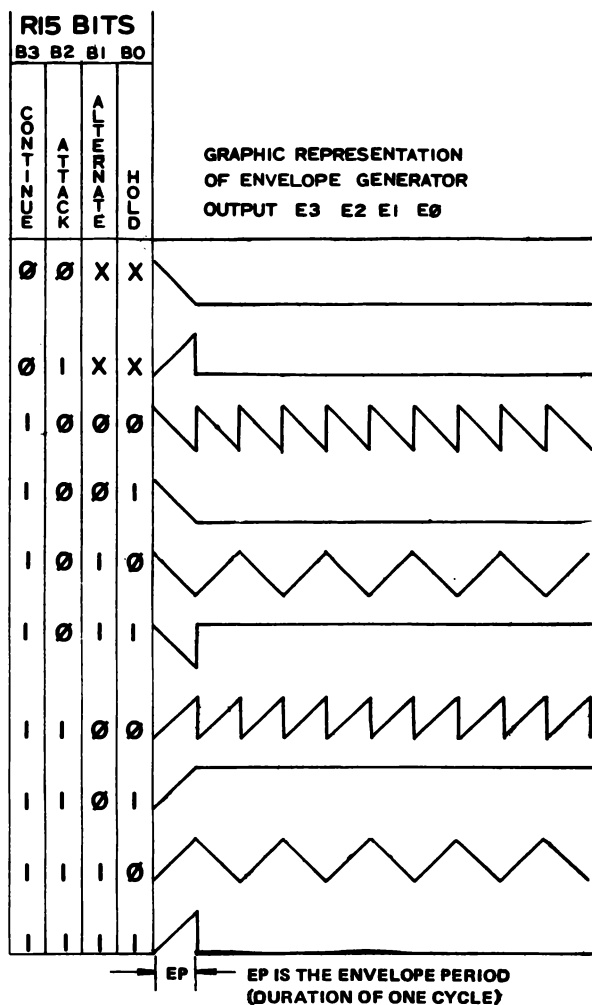
$$FT_{10} = 9 = 00001001 (B7-B0)$$



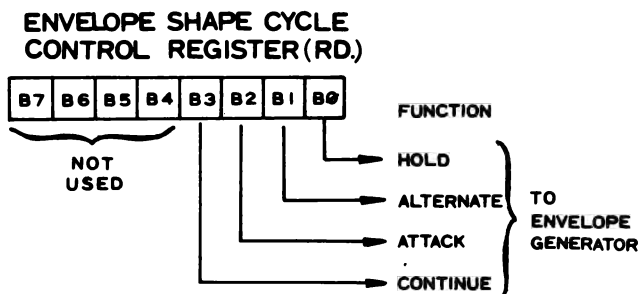
16 BIT ENVELOPE PERIOD(EP)
TO ENVELOPE GENERATOR

- (6) RD: Register D provides four bits to control the output waveform in combination with registers B, C.

See the following figure and chart:



ENVELOPE SHAPE/CYCLE OPERATION



(7) RE, RF: Each provides eight bits as a buffer. These registers do not have significant effect on the generation of music tones.

3.1.3 Software and Application

The three I/O ports are used to program the 16 registers. See the following table on their usage:

Address	Label	Function
C071	RD-8912	;CPU reads from 8912 registers specified by C073
C072	WR-8912	;CPU writes to 8912 registers specified by C073
C073	PONT-8912	;CPU points to one of the 16 8912 registers

The three I/O ports above are the locations of the interface for CPU and AY-81912.

Examples:

(1)

```
LDX    #$0FH      ;
LDA     #$06H      ;
STA     PONT-8912  ;CPU will use register 6(R6) to
                    ;control NOISE.
STX     WR-8912    ;write 0FH into register 6,
                    ;effecting
                    ;f noise=2MHZ/16*(0F)=8.3 KHZ
```

(2)

```
LDX     #$07H      ;
LDA     #$08H      ;
STA     PONT-8912  ;CPU will use register 8, (R8) to
                    ;control NOISE.

STX     WR-8912    ;write 07H into register 8,
                    ;effecting the amplitude of channel
                    ;to be 7.
```

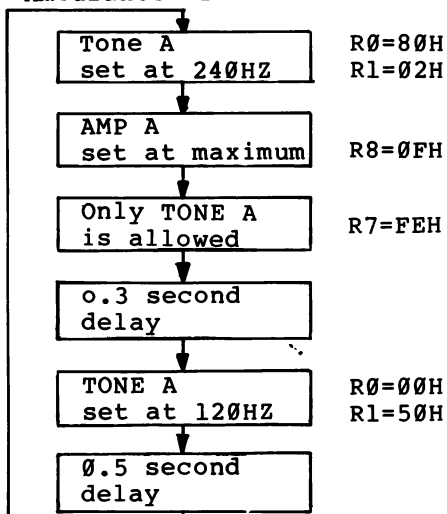
(3)

```
LDX     #$09H      ;
STA     PONT-8912  ;CPU will use register 9(R9) to
                    ;control the amplitude of channel B.
LDX     RD-8912    ;Read the R9 data into Rx of CPU, to
                    ;check the current amplitude of
                    ;channel B.
```

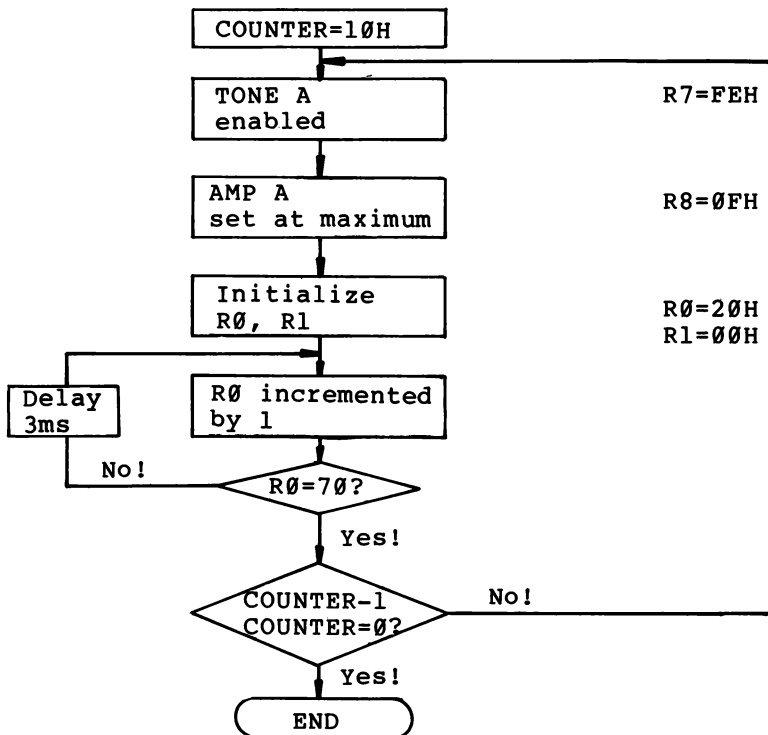
Note: To close a channel, you have to reset its AMP register. Otherwise, the channel will produce the sound effect with the frequency specified by Noise or Tone registers indefinitely.

(4) Below you will see the flowcharts of "Ambulance" and "Laser Gun". By following these steps to program registers, you can obtain the desired sound effects.

"Ambulance" Flowchart



"Laser Gun" Flowchart



To program the registers on the MPF-III, you have to set the values first and then call the SG routine in the monitor. The routines of the 16 registers are listed in the following table:

(CH stands for channel in the list)

Label	Input Parameter	Address (Hex) (Dec)	Function
A-TONE	Store X in Hi-byte Store Y in Low-byte	A00E 40974	set the fequency of CH.A Example: LDX #\$02 LDY #\$56 JSR A-TONE resulting in A=200H
B-TONE	Same as above	A017 40983	Set frequency of CH.B
C-TONE	Same as above	A020 40992	Set frequency of CH.C
NOISE	X	A029 41001	Set frequency of NOISE LDX #\$FH JSR NOISE resulting in NOISE=7.8 KHZ
ALLOW	X	A02D 41005	ENABLE select C B A C B A NOISE TONE Example: LDX #\$19 ;19H=00011001 JSR ALLOW Enabling CH.A generate both TONE and NOISE Enabling CH.B generate NOISE
A-AMP	X	A031 41009	Set amplitude of CH.A Example: LDX #\$08H JSR A-AMP Setting the volumn of AMP at medium

B-AMP	X	A035 41013	Set amplitude of CH.B
C-AMP	X	A039 41017	Set amplitude of CH.C
MODULATOR	Store X in Hi-byte Store Y in Low-byte	A03D 41021	Set time of amplitude attenuation when AMP register=10H (Bit 4=1)
SHAPE	X	A046 41030	Set output waveform when AMP register=10H (Bit 4=1)
PLAY-500ms	none	A05E 41054	Delay 0.5 second
Y-2ms	Y	A060 41056	Delay Y*2ms second
Y-ms	Y	A063 41059	Delay Y*ms second
SILENCE	none	A04A 41034	Silence the three channels of 8912
GUN-SHOT	none	A0E2 41186	Generate sound of machine gun for 0.5 second
CONT-GUN	Y	A105 41221	Generate sound of machine gun for Y*0.5 second
LASER	none	A0B3 41139	Generate 10 sounds of laser gun
LAL	Y	A0B5 41141	Generate Y sounds of laser gun
BOMB	none	A06C 41068	Generate sound of a dropping bomb
BO1	0CCH	A06E 41071	Generate sound of a dropping bomb; sound duration decided by the value at 0CCH
EXPLO	none	A092 41106	Generate sound of explosion
Label	Input Parameter	Address (Hex) (Dec)	Function
A-MUSIC	0CEH 0CFH	A4B8 42168	Autoplay music starting from the beginning point specified by (CFH) (CEH)
GO-ASM-BASS	X	A2DC 41692	X selects the kind of BASS (X=1,2,...8) or disables BASS (X=0)
BELL-PROCESSOR	none	A426 42022	Select BELL
PIANO-PROCESSOR	none	A42F 42031	Select PIANO
XYLOPHONE-PROCESSOR	none	A41D 42013	Select XYLOPHONE
ORGAN-PROCESSOR	none	A438 42040	Select ORGAN

3.1.4 Composing Music with the Sound Generator

Upon powering on, the MPF-III will initialize the tempo at a medium of 120 beats/min, and select the piano as the instrument by default. To change the tempo, you may use the following instructions:

DEC 02FBH ; Approximately 144 beats/min.

DEC 02FBH ; Approximately 180 beats/min.

DEC 02FBH ; Approximately 240 beats/min.

The above steps are effective only if the current tempo is set at the default 120 beats/min. Otherwise the following alternatives should be adopted:

```
LDX #$04H
STX 02FBH ;180 beats/min
```

```
or LDX #$03H
STX 02FBH ;240 beats/min
```

To change timbre (the quality of the sound), you may use the instrument-processor subroutines:

```
JSR PIANO-PROCESSOR ; Select the sound of piano
JSR BELL-PROCESSOR ; Select the sound of bell
JSR XYLOPHONE-PROCESSOR ; Select the sound of xylophone
JSR ORGAN-PROCESSOR ; Select the sound of organ
```

Below we will show you how to use the Sound Generator in an assembly program.

First store the music score of the "Janken song". in the memory starting from address 5000H

(Janken is A finger-flashing game of paper-scissors-stone)

G2/4 "Janken Song"

G

```
|| 5 1 | 1 3̣1 | 5 3 | 3 0 | 5 3 | 1.3̣ | 2 1 |
| 2 0 | 5 5 | 5.3̣ | 4 6 | 6 0 | 5 3 | 4 2 |
| 1 7̣ | 1 0 ||
```

```

5000H 15 75 31 75 31 75 33 73 31 73 15
      75 33 75 33 75
5010H 00 75 35 75 33 75 31 76 33 73 32
      75 31 75 32 75
5020H 00 75 35 75 35 75 35 76 33 73 34
      75 36 75 36 75
5030H 00 75 35 75 35 75 34 75 32 75 31
      75 17 75 31 75
5040H 00 75 FF FF

```

As shown in the above data lines, two FF are placed at the end. Lastly, store the beginning address of the music score into locations CEH and CFH to call the music subroutine and you can enjoy the musical piece.

```

LDX    #$00H    ; Specify the beginning address of
STX    CEH      ; the music score
LDX    #$50H    ;
STX    CFH      ;
JSR    A-MUSIC  ; Music begins playing until
                  encounters the end.

```

To add drum accompaniment in a song, refer to the table on Accompaniment and use the follow subroutine.

```

LDX    #$02H    ; select BASS 2 (=BLUES)
JSR    GO-ASM-BASS
LDX    #$00H
STX    CEH
LDX    #$50H
STX    CFH
JSR    A-MUSIC

```

To use a specific musical instrument for accompaniment, follow the steps listed below:

```

JSR    BELL-PROCESSOR    ;Select bell as the instrument
LDX    #$02H            ;Select blues as accompaniment
JSR    GO-ASM-BASS      ;
LDX    #$00H            ;Play "Janken Song" from 5000H
STX    CEH              ;
LDX    #$50H            ;
STX    CFH              ;
JSR    A-MUSIC          ;

```

The BASIC command PLAY may also be used to play a song. For example, the instruction

PLAY 20480, 6

will play the "Janken Song" starting from 5000H with a drum accompaniment of BLUES.

To compose music exclusively in BASIC commands, refer to related manual on MPF-III BASIC programming.

* A list of SG BASIC intructions:

SONG P1,P2 ;P1 for music notes
P2 for beats.

BASS P1 ;Selects accompaniment

TEMPO P1 ;Selects tempo

MUSIC P1 ;Selects instrument

PLAY P1,P2 ;Play music starting from location
P1 in instrument P2

EFFECT P1 ;Select special sound effect

** A summary of tables for reference **

Label	address (Hex)	Function
TEMPO-REF	02FBH	Selects Tempo, 02FB=06H selects Moderato
BASS-FLAG	02EEH	Enables/disables accompaniment

Table of beats

BEAT	$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{3}{4}$	1	$1\frac{1}{2}$	2	3	4
NOTE										
CODE	70	71	72	73	74	75	76	77	78	79

Table of accompaniments

Ø		STOP	
1	WALTZ	5	CHACHA
2	RUMBA	6	BLUES
3	TANGO	7	ROCK
4	MARCH	8	SWING

Table of tones

		DO .	RE .	MI .	FA .	SO .	LA .	SI .
LOW		11	12	13	14	15	16	17
TONE	#SHARP	21	22	*	24	25	26	*

		DO	RE	MI	FA	SO	LA	SI
MIDDLE		31	32	33	34	35	36	37
TONE	#SHARP	41	41	*	44	45	46	*

		DO .	RE .	MI .	FA .	SO .	LA .	SI .
HIGH		51	52	53	54	55	56	57
TONE	#SHARP	61	62	*	64	65	66	*

REST	MUSIC	END CODE
	BASIC	ASSEMBLY
00	100	FF

Table of Tone-Frequency Conversion

TONE	NOTE	OCTAVE	FREQUENCY	REGISTER VALUE	
				HEX.	DECIMAL
DO	C	LOW	261.624	1ACH	428
DO#	C#		277.184	194H	404
RE	D		293.664	17DH	381
RE#	D#		311.128	160H	352
MI	E		329.624	153H	339
FA	F		349.232	140H	320
FA#	F#		369.992	12EH	302
SO	G		391.992	11DH	285
SO#	G#		415.304	10DH	269
LA	A		440.000	0FEH	254
LA#	A#		466.160	0F0H	240
SI	B		493.880	0E2H	226
DO	C	MIDDLE	523.248	0D6H	214
DO#	C#		554.368	0CAH	202
RE	D		587.328	0BEH	190
RE#	D#		622.256	0B4H	180
MI	E		629.248	0AAH	170
FA	F		698.464	0A0H	160
FA#	F#		739.984	097H	151
SO	G		783.984	08FH	143
SO#	G#		830.608	087H	135
LA	A		880.000	07FH	127
LA#	A#		932.320	078H	120
SI	B		987.760	071H	113
DO	C	HIGH	1046.496	06BH	107
DO#	C#		1108.736	065H	101
RE	D		1174.656	05FH	95
RE#	D#		1244.512	05AH	90
MI	E		1318.496	055H	85
FA	F		1396.928	050H	80
FA#	F#		1479.968	04CH	76
SO	G		1567.968	047H	71
SO#	G#		1661.216	043H	67
LA	A		1760.000	040H	64
LA#	A#		1864.640	03CH	60
SI	B		1975.520	039H	57

Chapter 4

Memory Organization

The MPF-III's 6502 microprocessor can address 65,536 (64K) memory locations (K stands for 1024). All of the programmable storage (RAM, for random-access memory), read-only storage (ROM, for read-only memory) and input and output devices are allocated locations in this 64K address space. Some functions share the same addresses -- but not, at the same time. For information about these shared address spaces, see the section "Bank-switched Memory " in this chapter.

We refer to the MPF-III's memory in 256-byte blocks called pages. One reason for this is that a one-byte address counter or index register can specify one of 256 different locations. Thus, page 0 consists of memory locations from 0 to 255 (hexadecimal \$0 to \$FF), inclusive. Page 1 consists of locations 256 to 511 (hexadecimal \$100 to \$1FF -- note that the page number is the high-order part of the hexadecimal address). Don't confuse this kind of page with the display buffers in the MPF-III, which are sometimes referred to as Page 1 and Page 2.

4.1 Main Memory Map

The map of the main memory address space in Fig. 4-1 shows the functions of the major areas of memory. The bank-switched memory in the memory space from 52K to 64K (\$D000 through \$FFFF) is described below.

4.1.1 RAM Memory Allocation

As shown in Fig. 4-1, the major portion of the MPF-III's memory space is allocated to programmable storage (RAM). Fig. 4-2 shows the areas allocated to RAM. The main RAM memory extends from location 0 to location 49151 (hex \$BFFF), and occupies pages 0 through 191 (hexadecimal \$BF). There is also RAM storage in the bank-switched space from 53248 to 65535 (hexadecimal \$D000 to \$FFFF).

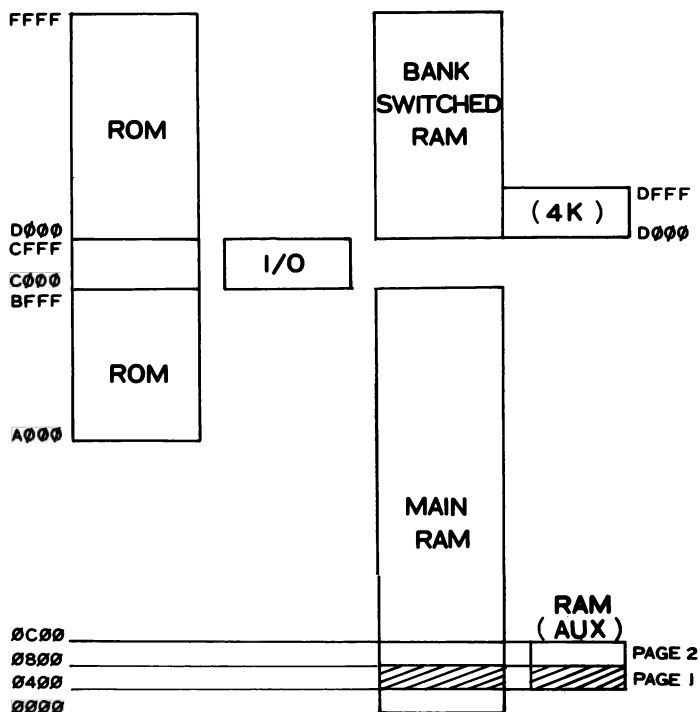


Fig. 4-1 MPF-III Memory Organization

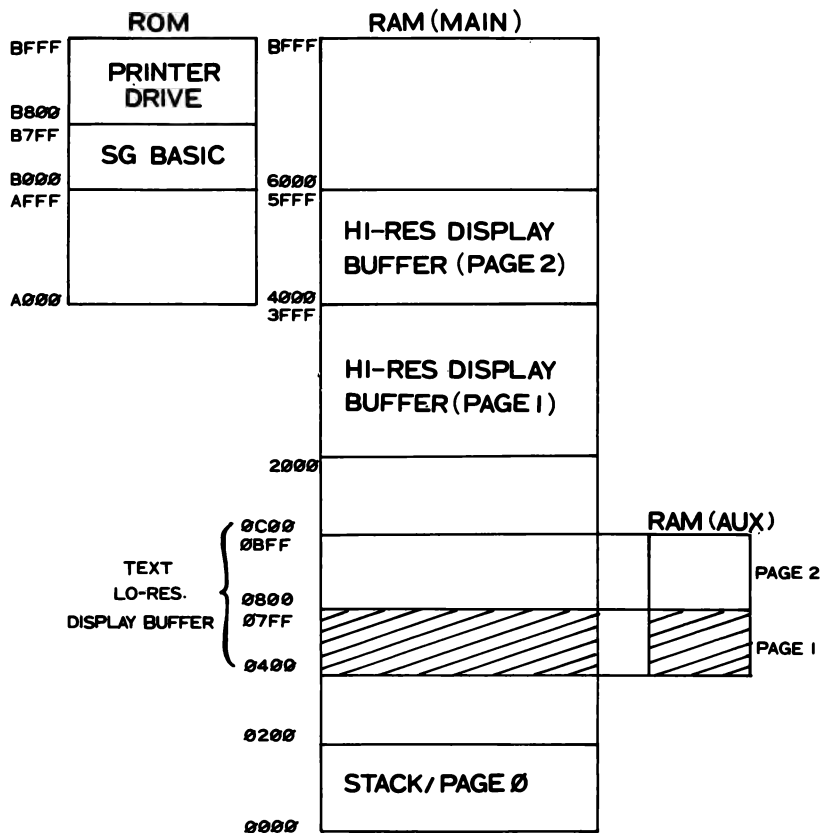


Fig. 4-2 RAM Allocation Map

Reserved Memory Pages

Most of the MPF-III's RAM is available for storing your programs and data. However a few RAM pages are reserved for the use of the Monitor firmware and the BASIC interpreters. The reserved pages are described below.

Note: The system does not prevent your using these pages, but if you do use them, you must be careful not to disturb the system data they contain, or you will cause the system to malfunction.

Page Zero

Several of the 6502 microprocessor's addressing modes require the use of addresses in page zero, also called zero page. The Monitor, the BASIC interpreters, and DOS all make extensive use of page zero.

To use indirect addressing in your assembly-language programs, you must store base addresses in page zero. At the same time, you must avoid interfering with the other programs that use page zero -- the Monitor, the BASIC interpreters, and the Disk Operating Systems. One way to avoid conflicts is to use only those page-zero location not already used by other programs. Fig. 4-3, 4-4 and 4-5 show the locations in page zero used by the Monitor, BASIC, and MPF-DOS.

High Nybble of Address	Low Nybble of Address															
	\$0	\$1	\$2	\$3	\$4	\$5	\$6	\$7	\$8	\$9	\$A	\$B	\$C	\$D	\$E	\$F
\$00																
\$10																•
\$20		•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$30		•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$40		•	•	•	•	•	•	•	•	•					•	•
\$50		•	•	•	•	•										
\$60																
\$70																
\$80																
\$90																
\$A0																
\$B0																
\$C0																
\$D0																
\$E0																
\$F0																

Fig. 4-3 Zero Page Locations used by Monitor

High Nybble of Address	Low Nybble of Address															
	\$0	\$1	\$2	\$3	\$4	\$5	\$6	\$7	\$8	\$9	\$A	\$B	\$C	\$D	\$E	\$F
\$00																
\$10																
\$20																
\$30																
\$40																
\$50																
\$60																
\$70																
\$80																
\$90																
\$A0																
\$B0																
\$C0																
\$D0																
\$E0																
\$F0																

Fig. 4-4 Zero Page Locations used by BASIC

High Nybble of Address	Low Nybble of Address															
	\$0	\$1	\$2	\$3	\$4	\$5	\$6	\$7	\$8	\$9	\$A	\$B	\$C	\$D	\$E	\$F
\$00	•	•	•	•	•	•					•	•	•	•	•	•
\$10	•	•	•	•	•	•	•	•	•	•				•		
\$20																
\$30																
\$40																
\$50	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$60	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$70	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$80	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$90	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$A0	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$B0	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$C0	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
\$D0	•	•	•	•	•	•			•	•	•	•	•	•	•	•
\$E0	•	•	•	•	•	•	•	•	•	•	•					
\$F0	•	•	•	•	•	•	•	•	•							

Fig. 4-5 Zero Page Locations used by MPF-DOS

As you can see from the figures page zero is almost all used up, except for a few bytes here and there, It's hard to find more than one or two bytes that aren't used by either BASIC or the Monitor or DOS. Rather than trying to squeeze your data into the scattered free locations you should prefer a safer alternative: save the contents of part of page zero, use that part, then restore the previous contents before you pass control to another program.

4.1.2 The 6502 Stack

The 6502 microprocessor uses page 1 as the stack -- the place where subroutine return addresses are stored, in first-in, last-out sequence. Many programs also use the stack for temporary storage of the registers (via push and pull operation.) You can do the same, but you should use it sparingly. The stack pointer is eight bits long, so the stack can hold only 256 bytes of information at a time. When you store the 257th byte in the stack, the stack pointer repeats itself, or wraps around, so that the new byte replaces the first byte stored, which is now lost. This writing over old data is called stack overflow, and when it happens, the program continues to run normally until the lost information is needed, whereupon the program terminates catastrophically.

4.1.3 The Input Buffer

The RDLIN input routine, which is used by the Monitor and the BASIC interpreters, uses page 2 as its keyboard-input buffer. The size of this buffer sets the maximum size of input strings.

(Note: The MPF-III BASIC only uses the first 237 bytes, although it permits you to type all 256). If you know that you won't be typing any long input strings, you can store temporary data at the upper end of page 2.

Display Page 2, the alternate text and low-resolution-graphics display buffer, occupies memory pages 8 through 11 (locations 2048 through 3071, hexadecimal \$0800 through \$0BFF). Most programs do not use Page 2 for displays, so you can use this area for program or data storage. They are not for screen display.

4.1.4 Link-address storage

The Monitor and MPF-DOS both use the upper part of page 3 for link addresses or vectors. The following table shows the part of page 3 used by the Monitor.

Vector Address		Vector Function
Decimal	Hex	
1008	\$3F0	Address of the subroutine that handles BRK requests
1009	\$3F1	
1010	\$3F2	Reset vector
1011	\$3F3	
1012	\$3F4	Power-up byte
1013	\$3F5	Jump instruction to the subroutine that handles MPF-III "&" commands
1014	\$3F6	
1015	\$3F7	
1016	\$3F8	Jump instruction to the subroutine that handles user (CTRL-Y) commands
1017	\$3F9	
1018	\$3FA	
1019	\$3FB	Jump instruction to the subroutine that handles non-maskable interrupts
1020	\$3FC	
1021	\$3FD	
1022	\$3FE	Interrupt vector (address of the subroutine that handles interrupt requests)
1023	\$3FF	

BASIC programs sometimes need short machine-language routines. These routines are usually stored in the lower part of page 3.

4.1.5 The Display Buffers

The primary text and low-resolution graphics display buffer occupies memory pages 4 through 7 (locations 1024 through 2047, hexadecimal \$0400 through \$07FF). This entire 1024-byte area is called display Page 1, and it is not usable for program and data storage. 64 locations in this buffer are reserved for use by the peripheral cards.

The primary high-resolution graphics display buffer, called high-resolution Page 1, occupies memory pages 32 through 63 (locations 8192 through 16383, hexadecimal \$2000 through \$3FFF). If your program doesn't use high-resolution graphics, this area is usable for programs or data.

High-resolution graphics Page 2 occupies memory pages 64 through 95 (locations 16384 through 24575, hexadecimal \$4000 through \$5FFF). Most programs use this area for program or data storage.

4.2 Bank-switched Memory

The memory address space from 52K to 64K (hexadecimal \$D000 through \$FFFF) is doubly allocated: it is used for both ROM and RAM. The 12K bytes of ROM (read-only memory) in this address space contain the Monitor and the BASIC interpreter. Alternatively, there are 16K bytes of RAM for this space. You may use them for specific purposes.

You may also be wondering how 16K bytes of RAM is mapped into only 12K bytes of address space. The answer is that the 4K-byte address space from 52K to 56K (hexadecimal \$D000 through \$DFFF) is used twice.

Switching different blocks of memory into the same address space is called bank switching. There are three examples of bank-switching going on here: first, the entire address space from 52K to 64K (\$D000 through \$FFFF) is switched between ROM and RAM, and second, the address space from 52K to 56K (\$D000 to \$DFFF) is switched between two different blocks of RAM. Lastly, the address space from 40K to 48K (\$A000 to \$BFFF) is switched between ROM and RAM.

4.2.1 Setting Bank Switches

You switch bank of memory in the same way you switch other functions in the MPF-III: by using soft switches. These soft switches do three things: select either RAM or ROM in this memory space; enable or inhibit writing to the RAM (write-protect); and select the first or second 4K-byte bank of RAM in the address space \$D000 to \$DFFF.

NOTE: Do not use these switches without careful planning. Careless switching between RAM and ROM is almost certain to have catastrophic effects on your program.

Fig. 4-6 shows the addresses of the soft switches for enabling all combinations of reading and writing in this memory space. All of the hexadecimal values of the addresses are of the form \$C08x. Notice that several addresses perform the same function: this is because the soft switches are activated by combinations of address bits. For example, any address of the form \$C08x with a 1 in the low-order bit enables the RAM for writing. Similarly, bit 3 of the address selects which 4K block of RAM to use for the address space \$D000-\$DFFF; if bit 3 is 0, the second bank of RAM is used, and if bit 3 is 1, the first bank is used.

Switch Address	Write RAM	Read RAM	Read ROM	4K RAM Bank:		Notes
				First	Second	
\$C080		•			•	
\$C081	•		•		•	1
\$C082			•		•	
\$C083	•	•			•	2
\$C084		•			•	
\$C085	•		•		•	1
\$C086			•		•	
\$C087	•	•			•	2
\$C088		•		•		
\$C089	•		•	•		1
\$C08A			•	•		
\$C08B	•	•		•		2
\$C08C		•		•		
\$C08D	•		•	•		1
\$C08E			•	•		
\$C08F	•	•		•		2

Fig. 4-6 Bank Select Switches

When RAM is not enabled for reading, the ROM in this address space is enabled. Even when RAM is not enabled for reading, it can still be written to if it is write-enabled.

When you turn power on or reset the MPF-III, it initializes the bank switches for reading the ROM and writing the RAM, using the second bank of RAM. This does not affect the bank-switched memory.

When SELROMAB is active high, the ROM space from 40K to 48K is enabled. When SELROMAB is active low, the RAM space is enabled. Consult the following table.

Name	Function	Location	Note
SELROMAB	Select ROMAB off	\$C078	write
	Select ROMAB on	\$C079	write
	Read SELROMAB switch	\$C022	read

As you can see from the following chart, the auxiliary memory is divided into two sections. The first section is the one from 1024 through 2047 (\$0400 through \$07FF).

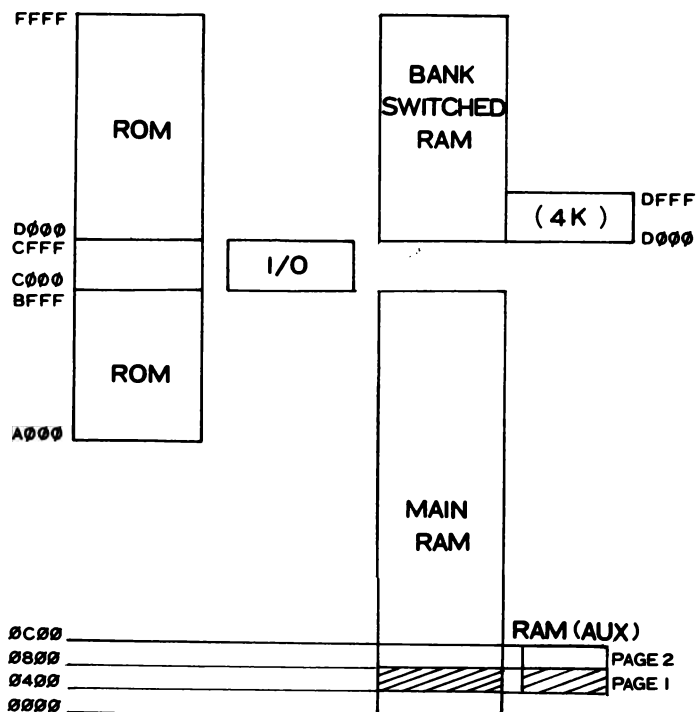


Fig. 4-7 MPF-III Memory Organization

The other section of the auxiliary memory ranges from 2048 through 3071 (\$0800 through \$0BFF). If you use the auxiliary RAM in this space, the soft switches have the same effect on the auxiliary RAM that they do on the main RAM.

Name	FUNCTION	Location		Notes
		Hex	Decimal	
RAMRD	READ auxiliary memory	\$C003	49155	Write
	Read main memory	\$C002	49154	Write
	Read RAMRD switch	\$C013	49171	Read
RAMWRT	Write auxiliary memory	\$C005	49157	Write
	Write main memory	\$C004	49156	Write
	Read RAMWRT switch	\$C014	49172	Read
80STORE	On: store in aux. memory	\$C001	49153	Write
	Off: store in main memory	\$C000	49152	Write
	Read 80STORE switch	\$C018	49176	Read
PAGE 2	Page 2 on (Aux. memory)	\$C055	49237	
	Page 2 off (Main memory)	\$C054	49236	
	Read PAGE 2 switch	\$C01C	49180	Read
HIRES	On: access HIRES pages	\$C057	49239	
	Off: HIRES off	\$C056	49238	
	READ HIRES switch	\$C01D	49181	Read
AUXPG	Auxiliary display page 1	\$C07A	49274	Write
	Auxiliary display page 2	\$C07B	49275	Write
	Read AUXPG switch	\$C023	49187	Read

4.3 Reset Routine

The reset routine is built into the MPF-III firmware, and it is initiated any time you turn on the power or press the RESET key while holding down the CONTROL key. The reset routine puts the MPF-III into its normal operating mode and restarts the resident program.

When you initiate a reset, hardware in the MPF-III sets the memory-controlling soft switches to normal: main RAM and ROM are enabled, and auxiliary RAM is disabled and the bank-switched memory space is set up to read ROM and write to RAM, using the second bank at \$D000.

The reset routine sets the display-controlling soft switches to display 40-column text Page 1, then sets the window equal to the full 40-column display, puts the cursor at the bottom of the screen and sets the display format to normal.

The reset routine sets the keyboard and display as the standard input and output devices by loading the standard I/O links. It turns annunciators 0 and 1 off and annunciators 2 and 3 on, clears the keyboard strobe and outputs a bell (tone).

The MPF-III has three types of reset: power-on reset, also called cold-start reset; warm-start reset; and forced cold-start reset. The procedure described above is the same for any type of reset. What happens next depends on the vector. The reset routine checks the reset vector to determine whether it is valid or not. If the reset was caused by turning the power on, the vector will not be valid, and the reset routine will perform the cold-start procedure. If the vector is valid, the routine will perform the warm-start procedure.

4.3.1 The Cold-start Procedure

If the reset vector is not valid either because the MPF-III has just been turned on or something has caused memory contents to be changed, the reset routine clears the display and puts the string "MPF-III" at the top of the video display. It loads the reset vector and the validity-check byte, then starts checking the expansion slots to see if there is a disk drive controller card in one of them. If there is a controller card, it initiates the card's firmware. The bootstrap then loads the Disk Operating System from the disk drive. When DOS has been loaded, it displays other messages on the screen.

If the reset routine doesn't find a controller card, or if you press CTRL-RESET again before the startup procedure has been completed, the reset routine will continue without using the disk, and pass control to the built-in MPF-III interpreter.

4.3.2 The Warm-start Procedure

Whenever you press CTRL-RESET when the MPF-III has already completed a cold-start reset, the reset vector is still valid and it is not necessary to reinitialize the entire system. The reset routine simply uses the vector to transfer control to the resident program, which is normally the built-in BASIC interpreter. If you are using DOS, it is the resident program and it restarts BASIC, which you were using when you pressed CTRL-RESET.

4.3.3 Forced Cold-start

If a program has loaded the reset vector to point to the beginning of the program, pressing CTRL-RESET causes a warm-start reset that uses the vector to transfer control to that program. If you want to stop such a program without turning the power off and on, you can force a cold-start reset by holding down the CTRL-RESET and PB0 keys, then releasing the CTRL-RESET key first and then the PB0.

When you want to stop a program unconditionally - for example, to start up the MPF-III with some other program - you should use the forced cold-start reset, instead of turning the power off and on.

4.3.4 The Reset Vector

When you reset the MPF-III, the reset routine transfers control to the resident program by means of an address stored in page 3 of main RAM. This address is called a vector because it directs program control to a specified destination. There are several other vector addresses stored in page 3, including the interrupt vectors and the DOS vectors.

The cold-start reset routine stores the starting address of the built-in MPF-III interpreter, low-order byte first, in the reset vector address at locations 1010 and 1011 (hexadecimal \$3F2 and \$3F3). It then stores a validity-check byte, also called the power-up byte, at location 1012 (hexadecimal \$3F4). The validity-check byte is computed by performing an exclusive-OR of the second byte of the vector with the constant 165 (hexadecimal \$A5). Each time you reset the MPF-III, the reset routine uses this byte to determine whether the reset vector is still valid.

You can change the reset vector so that the reset routine will transfer control to your program instead of to the BASIC interpreter. For this to work, you must also change the validity-check byte to the exclusive-OR of the high-order byte of your new reset vector with the constant 165 (\$A5). If you fail to do this, then the next time you reset the MPF-III, the reset routine will determine that the reset vector is invalid and perform a cold-start reset, eventually transferring control to the disk bootstrap routine or to BASIC.

The reset routine has a subroutine that generates the validity-check byte for the current reset vector. You can use this subroutine by doing a subroutine call to location - 1169 (hexadecimal \$FB6F). When your program finishes, it can return the MPF-III to normal operation by restoring the original reset vector and again calling the subroutine to fix up the validity-check byte.

4.4 Peripheral Card Memory Space

As the 6502 CPU of MPF-III handles all the I/O operations through the memory locations, part of the MPF-III memory space are allocated for the exclusive use of the peripheral card on the expansion slot. The memory space allocated to the peripheral cards are used in the small dedicated programs such as I/O driver. The peripheral cards with its own driver programs in the firmware are called intelligent peripheral devices. They enable you to add peripheral hardware without changing the programs if your programs handle I/O data in normal conditions.

4.4.1 Peripheral Card I/O Space

Each expansion slot has the exclusive use of sixteen memory locations for data input and output in the memory space beginning at location \$C090. Slot 1 uses locations \$C090 through \$C09F, slot 2 uses locations \$C0A0 through \$C0AF, and so on through location \$C0FF.

These memory locations are used for different I/O functions, depending on the design of each peripheral card. Whenever the MPF-III addresses one of the sixteen I/O locations allocated to a particular slot, the signal on pin 41 of that slot, called DEVICE SELECT, switches to the active (low) state. This signal can be used to enable logic on the peripheral card that uses the four low-order address lines to determine which of its sixteen I/O locations is being accessed.

4.4.2 Peripheral Card ROM Space

One 256-byte page of memory space is allocated to each peripheral card. This space is normally used for read only memory on the card with driver programs that control the operation of the peripheral device connected to the card.

The page of memory allocated to each expansion slot begins at location \$Cn00, where n is the slot number, as shown in the following tables. Whenever the MPF-III addresses one of the 256 ROM memory locations allocated to a particular slot, the signal on pin 1 of that slot, called I/O SELECT, switches to the active (low) state. This signal enables the ROM or PROM devices on the card, and the eight low-order address lines determine which of the 256 memory locations is being accessed.

Slot	Locations	Enabled by
1	\$C100-\$C1FF	I/O SELECT'
2	\$C200-\$C2FF	I/O SELECT'
3	\$C300-\$C3FF	I/O SELECT'
4	\$C400-\$C4FF	I/O SELECT'
5	\$C500-\$C5FF	I/O SELECT'
6	\$C600-\$C6FF	I/O SELECT'
7	\$C700-\$C7FF	I/O SELECT'

Base Address	Slot Number						
	1	2	3	4	5	6	7
\$0478	\$0479	\$047A	\$047B	\$047C	\$047D	\$047E	\$047F
\$04F8	\$04F9	\$04FA	\$04FB	\$04FC	\$04FD	\$04FE	\$04FF
\$0578	\$0579	\$057A	\$057B	\$057C	\$057D	\$057E	\$057F
\$05F8	\$05F9	\$05FA	\$05FB	\$05FC	\$05FD	\$05FE	\$05FF
\$0678	\$0679	\$067A	\$067B	\$067C	\$067D	\$067E	\$067F
\$06F8	\$06F9	\$06FA	\$06FB	\$06FC	\$06FD	\$06FE	\$06FF
\$0778	\$0779	\$077A	\$077B	\$077C	\$077D	\$077E	\$077F
\$07F8	\$07F9	\$07FA	\$07FB	\$07FC	\$07FD	\$07FE	\$07FF

4.4.3 Expansion ROM Space

In addition to the small areas of ROM memory allocated to each expansion slot, peripheral cards can use the 2K-byte memory space from \$C800 to \$CFFF for larger programs in ROM or PROM. This memory space is called expansion ROM space. Besides being larger, the expansion ROM memory space is always at the same locations regardless of which slot is occupied by the card, making programs that occupy this memory space easier to write.

This memory space is available to any peripheral card that needs it. More than one peripheral card can have expansion ROM on it, but only one of them can be active at a time.

Each peripheral card that uses expansion ROM must have a circuit on it to enable the ROM. The circuit does this by a two-stage process: first, it sets a flip-flop when the I/O SELECT' signal, pin 1 on the slot, becomes active (low); second, it enables the expansion ROM devices when the I/O STROBE' signal, pin 20 on the slot, becomes active (low). The following figure shows a typical ROM-enable circuit.

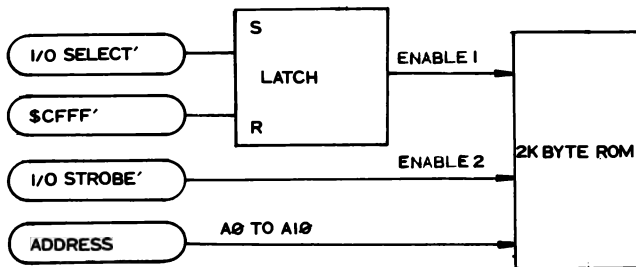


Fig. 4-8 Expansion ROM Enable Circuit

The I/O SELECT' signal on a particular slot becomes active whenever the MPF-III microprocessor addresses a location in the 256-byte ROM address space allocated to that slot. The I/O STROBE' signal on all of the expansion slots becomes active (low) when the 6502 addresses a location in the expansion-ROM memory space, \$C800-\$CFFF. The I/O STROBE' signal is used to enable the expansion-ROM devices on a peripheral card.

A program on a peripheral card can get exclusive use of the expansion ROM memory space by referring to location \$CFFF in its initialization phase. This location is special: all peripheral cards that use expansion ROM must recognize a reference to \$CFFF as a signal to reset their ROM-enable flip-flops and disable their expansion ROMs. Of course, doing so also disables the expansion ROM on the card that is about to use it, but the next instruction in the initialization code set the flip-flop on the expansion-ROM enable circuit on the card. Once this has been done, this card has exclusive use of the expansion memory space and its program can jump directly into the expansion ROM.

As described above, the expansion-ROM disable circuit resets the enable flip-flop whenever the 6502 addresses location \$CFFF.

To do this, the peripheral card must detect the presence of \$CFFF on the address bus. You can use the I/O STROBE' signal for part of the address decoding, since it is active for addresses from \$C800 through \$CFFF. If you can afford to sacrifice some ROM space, you can simplify the address decoding even further and save circuitry on the card. For example, if you give up the last 256 bytes of expansion ROM space, your disable circuit only needs to detect addresses of the form \$CFxx, and you can use the minimal disable-decoding circuitry shown in the following figure.

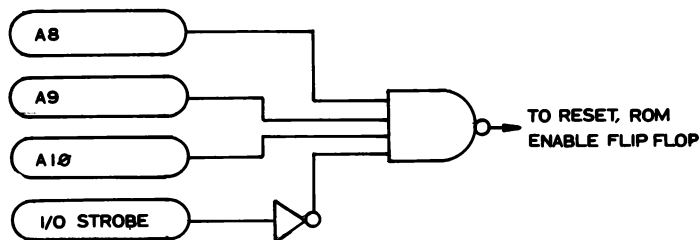


Fig. 4-9 ROM Disable Address Decoding

4.4.4 Peripheral Card RAM Space

There are 56 bytes of main memory allocated to the peripheral cards, eight bytes per card, as shown in the previous table. These 56 locations are actually in the RAM memory reserved for the text and low-resolution graphics displays, but these particular locations are not displayed on the screen and their contents are not changed by the built-in output routine CONOT1. Programs

in ROM on peripheral cards use these locations for temporary data storage.

* I/O Programming Suggestions

A program in ROM on a peripheral card should work no matter which slot the card occupies. If the program includes a jump to an absolute location in one of the 256-byte memory spaces, then the card will only work when it is plugged into the slot that uses that memory space. If you are writing the program for a peripheral card that will be used by many people, you should avoid placing such a restriction on the use of the card.

The first thing a peripheral-card subroutine should do is to save the contents of the 6502's registers. You can do this by using the monitor subroutine RGSAVE. This subroutine, which starts at location \$FF4A, stores the registers in zero-page memory locations \$45 - \$49. A companion subroutine, RGRSTD, restores the registers from these memory locations. Your program should call RGRSTD, which starts at location \$FF3F, just before it returns control to the program that called it.

This method of saving the registers is convenient, but not always safe. If a second subroutine calls RGSAVE, or if an interrupt occurs, the new register contents get saved in the same locations, and the old ones get destroyed. It is safer, though somewhat slower, to save the registers on the stack, and restore them just before returning control to the calling program.

Most single-character I/O is done via the 6502's accumulator. A character being output through your subroutine will be in the accumulator with its high bit set when your subroutine is called. Likewise, if your subroutine is performing character input, it must leave the character in the accumulator with its high bit set when it returns to the calling program.

* Finding the Slot Number

The memory addresses used by a program on a peripheral card differ depending on which expansion slot the card is installed in. Before it can refer to any of those addresses, the program must somehow determine the correct slot number. One way to do this is to execute a JSR (Jump to Subroutine) to a location with an RTS (Return from Subroutine) instruction in it, and then derive the slot number from the return address saved on the stack, as shown in the following example.

```

PHP           ; save status
SEI           ; inhibit interrupts
JSR $FF58     ; ▶ a known RTS instruction
TSX          ; get high byte of the ...
LDA $0100,X   ; ...return address from stack
AND #$0F      ; low-order digit is slot no.
PLP           ; restore status

```

* I/O Addressing

Once your peripheral-card program has the slot number, it can use it to address the I/O locations allocated to the slot. The following table shows how these locations are related to sixteen base addresses starting with \$C080. Notice that the difference between the base address and the desired I/O location has the form \$n0, where n is the slot number. Starting with the slot number in the accumulator.

Base Address	Connector Number						
	1	2	3	4	5	6	7
\$C080	\$C090	\$C0A0	\$C0B0	\$C0C0	\$C0D0	\$C0E0	\$C0F0
\$C081	\$C091	\$C0A1	\$C0B1	\$C0C1	\$C0D1	\$C0E1	\$C0F1
\$C082	\$C092	\$C0A2	\$C0B2	\$C0C2	\$C0D2	\$C0E2	\$C0F2
\$C083	\$C093	\$C0A3	\$C0B3	\$C0C3	\$C0D3	\$C0E3	\$C0F3
\$C084	\$C094	\$C0A4	\$C0B4	\$C0C4	\$C0D4	\$C0E4	\$C0F4
\$C085	\$C095	\$C0A5	\$C0B5	\$C0C5	\$C0D5	\$C0E5	\$C0F5
\$C086	\$C096	\$C0A6	\$C0B6	\$C0C6	\$C0D6	\$C0E6	\$C0F6

\$C087	\$C097	\$C0A7	\$C0B7	\$C0C7	\$C0D7	\$C0E7	\$C0F7
\$C088	\$C098	\$C0A8	\$C0B8	\$C0C8	\$C0D8	\$C0E8	\$C0F8
\$C089	\$C099	\$C0A9	\$C0B9	\$C0C9	\$C0D9	\$C0E9	\$C0F9
\$C08A	\$C09A	\$C0AA	\$C0BA	\$C0CA	\$C0DA	\$C0EA	\$C0FA
\$C08B	\$C09B	\$C0AB	\$C0BB	\$C0CB	\$C0DB	\$C0EB	\$C0FB
\$C08C	\$C09C	\$C0AC	\$C0BC	\$C0CC	\$C0DC	\$C0EC	\$C0FC
\$C08D	\$C09D	\$C0AD	\$C0BD	\$C0CD	\$C0DD	\$C0ED	\$C0FD
\$C08E	\$C09E	\$C0AE	\$C0BE	\$C0CE	\$C0DE	\$C0EE	\$C0FE
\$C08F	\$C09F	\$C0AF	\$C0BF	\$C0CF	\$C0DF	\$C0EF	\$C0FF

* RAM Addressing

A program on a peripheral card can use the eight base addresses shown in the preceding table to access the eight RAM locations allocated for its use. The program does this by putting its slot number into the Y index register and using indexed addressing mode with the base addresses. The base addresses can be defined as constants because they are the same no matter which slot the peripheral card occupies.

If you start with the correct slot number in the accumulator (by using the example shown earlier), the following example uses all eight RAM locations allocated to the slot.

```
TAY
LDA    $0478,Y
STA    $04F8,Y
LDA    $0578,Y
STA    $05FB,Y
LDA    $0678,Y
STA    $06F8,Y
LDA    $0778,Y
STA    $07FB,Y
```

* Changing the Standard I/O Links

There are two pairs of locations in the MPF-III that are used for controlling character input and output. They are called the I/O links. In an MPF-III running without a Disk Operating System, the I/O links normally contain the starting addresses of the standard input and output routine KBDIN and CONOT 1. If a disk operating system is running, one or both of the links will hold the addresses of the DOS input and output routines.

The link at locations \$36 and \$37 (decimal 54 and 55) is called CSW, for Character output Switch. Individually, location \$36 is called CSWL (CSW Low) and location \$37 is called CSWH (CSW High). This link holds the starting address of the subroutine the MPF-III is currently using for single-character output. This address is normally \$FDF0, the address of routine CONOT 1.

When you issue a PR#n from BASIC or an n CTRL-P from the Monitor, the MPF-III changes this link address to the first address in the ROM memory space allocated to slot number n. That address has the form \$Cn00. Subsequent calls for character output are thus transferred to the program on the peripheral card. That program can use the instruction sequences given above to find its slot number and use the I/O and RAM locations allocated to it. When it is finished the program can execute an RTS (Return from Subroutine) instruction to return control to the calling program, or jump to the output routine CONOT 1 at location \$FDF0 to display the output character (which must be in the accumulator) on the screen, then let CONOT1 return to the calling program.

A similar link at locations \$38 and \$39 (decimal 56 and 57) is called KSW, for Keyboard input Switch. Individually, location \$38 is called KSWL (for KSW Low) and location \$39 is called KSWH (KSE High). This link holds the starting address of the routine currently being used for single-character input. This address is normally \$FD1B, the starting address of the standard input routine KBDIN.

When you issue an IN#n command from BASIC or an n CTRL-K from the monitor, the MPF-III changes this link address to \$Cn00, the beginning of the ROM memory space that is allocated to slot number n. Subsequent calls for character input are thus transferred to the program on the peripheral card. That program can use the instruction sequences given above to find its slot number and use the I/O and RAM locations allocated to it. The program should put the input character, with its high bit set, into the accumulator and execute an RTS (Return from Subroutine) instruction to return control to the program that requested input.

When the Disk Operating System (DOS) is running, one or both of the standard I/O links hold addresses of the Disk Operating System's input and output routines. The DOS has internal locations that hold the addresses of the character input and output routines that are currently active.

If a program that is running with DOS changes the standard link addresses, either directly or via IN# and PR# commands, DOS is disconnected from the system.

To avoid disconnecting DOS each time they initiate I/O to a slot, BASIC programs that run with DOS must always issue an IN# or a PR# command from inside a PRINT statement that starts with a CTRL-D character. For assembly-language programs, there is a DOS subroutine call to use when changing the link addresses. After changing CSW or KSW, the program calls this subroutine at location \$3EA (decimal 1002). The subroutine transfers the link address to a location inside DOS and then restores the DOS address in the standard link location.

* Using Interrupts

Although programs running on the MPF-III do not normally use interrupts, it is possible to do so. To use interrupts on the MPF-III, your peripheral card must be able to send an interrupt vector, as described below.

Interrupt priority is handled by a daisy-chain arrangement using two pins, INT IN and INT OUT, on each expansion slot. On peripheral cards that don't use interrupts, these pins should be connected together. The daisy chain gives priority to the peripheral card in slot 7: if this card opens the connection between INT IN and INT OUT, or if there is no card in this slot, interrupt requests from cards in slots 1 through 6 can't get through. Similarly, slot 6 controls IRQ from slots 1 through 5, and so on down the line.

When the IRQ line on the 6502 microprocessor is activated (pulled low), the 6502 transfers control through the vector in locations \$FFFE-\$FFFF. This vector is the address of the Monitor's interrupt handler, which determines whether the request is due to an external IRQ or BRK instruction and transfers control to the appropriate routine via the vectors stored in memory page 3. The BRK vector is in locations \$3F0-\$3F1 and the IRQ vector is in locations \$3FE-\$3FF. The Monitor normally stores the address of its reset routine in the IRQ vector; you should substitute the address of your program's interrupt-handling routine.

* Other Uses of I/O Memory Space

The portion of memory space from location \$C000 through \$CFFF (decimal 49152 through 53247) is normally allocated to I/O and program memory on the peripheral cards, the 80-column display firmware also uses this memory space.

* Switching I/O Memory

The built-in firmware uses two soft switches to control the allocation of the I/O memory space from \$C000 to \$CFFF. The locations of these soft switches, INTCXROM, are given in the following table.

Name	Function	Location		Note
		Hex	Decimal	
INTCXROM	Internal ROM at \$Cx00	\$C007	49159	Write
	Slot ROM at \$Cx00	\$C006	49158	Write
	Read INTCXROM switch	\$C015	49173	Read

When INTCXROM is inactive (low), the I/O memory space from \$C100 to \$C7FF is allocated to the expansion slots. Setting SLOTXROM active (high) disables the peripheral-card ROM and selects built-in ROM in all of the I/O memory space except the part from \$C000 to \$C0FF.

Chapter 5

Input/Output

5.1 I/O Devices

In this chapter we will describe on the I/O devices of the MPF-III. Below is a list of the I/O devices:

- 1) Keyboard
- 2) 80-column
- 3) Video-display generator
- 4) Speaker
- 5) Cassette input and output
- 6) Game input and output
- 7) Printer interface
- 8) Sound generator

5.1.1 Keyboard

The MPF-III keyboard decoder can generate 232 ASCII codes(\$00-\$06, \$1F-\$FF), To get a character code through the keyboard, a machine language program reads a byte from the keyboard data location. Consult the following table:

LOCATION		DESCRIPTION
HEX	DECIMAL	
\$C000	49152	Keyboard data and strobe
\$C010	49168	Any-key-down flag and clear strobe switch

The low order 7 bits (bit 0-bit 6) constitutes the character code of the byte at the keyboard data location. The high order bit (bit 7) is the strobe bit. Your program can detect if any key (except for PB0 and PB1) is pressed by reading the byte at address 49168. Bit 7 is called Any-key-down, if a key is pressed, its value becomes 1, otherwise it is 0. The bit has a value of 128, therefore if you use the PEEK function to get the value of the byte, you will find it is equal to or greater than 128 if a key is pressed, otherwise, it is less than 128.

The high order bit of the keyboard data byte is the strobe bit. If a key is pressed, the strobe bit will remain high until we read or write the clear strobe location to reset it. this location is a combination of flag and switch. The flag will detect if a key is pressed, and the switch will clear the strobe bit. The switch function of this memory location is called a soft switch. You will reset the keyboard strobe whether you read or write this location.

NOTE: Whenever you read Any-key-down flag, you will clear the keyboard strobe at the same time. Therefore, if you want to read flag and strobe in a program, you must read the strobe bit at first.

Once the keyboard strobe is cleared, it will remain low until any key is pressed. Even if you clear the strobe, you can read the character code at the keyboard data location. This is because the low order 7 bits will remain the same until a key is pressed.

The codes generated when the keys are pressed are listed in the following tables. The codes from 80 to FF except 83 and 93 have the auto-repeat feature. If you press these keys for more than 0.9 second, the corresponding code will be sent out continuously at a rate of 15/second.

NOTE: In the binary key representation on the top of the following table, the three bits are used to show the status of ALT, the shift key , and the control key (CTRL). It is 1 if the corresponding key is pressed, otherwise it is 0.)

Tables of ASCII Character Codes

KEY	ALT/ ↑ /CTRL							
	000	001	010	011	100	101	110	111
0 INSC	00	00	B0	B0	00	00	B0	B0
1 CLRL	01	01	B1	B1	01	01	B1	B1
2 ↓	8A	8A	B2	B2	8A	8A	B2	B2
3 CPES	02	02	B3	B3	02	02	B3	B3
4 ←	88	88	B4	B4	88	88	B4	B4
5	B5	B5	B5	B5	B5	B5	B5	B5
6 →	95	95	B6	B6	95	95	B6	B6
7 CLRS	03	03	B7	B7	03	03	B7	B7
8 ↑	8B	8b	B8	B8	8B	8B	B8	B8
9 HOME	04	04	B9	B9	04	04	B9	B9
. DELC	05	05	AE	AE	05	05	AE	AE
+ RUN	06	06	AB	AB	06	06	AB	AB
- LIST	1F	1F	AD	AD	1F	1F	AD	AD
HALT	93	93	93	93	93	93	93	93
BREAK	83	83	83	83	83	83	83	83

F1	20	2C	38	44	50	50	50	50
F2	21	2D	39	45	51	51	51	51
F3	22	2E	3A	46	52	52	52	52
F4	23	2F	3B	47	53	53	53	53
F5	24	30	3C	48	54	54	54	54
F6	25	31	3D	49	55	55	55	55
F7	26	32	3E	4A	56	56	56	56
F8	27	33	3F	4B	57	57	57	57
F9	28	34	40	4C	58	58	58	58
F10	29	35	41	4D	59	59	59	59
F11	2A	36	42	4E	5A	5A	5A	5A
F12	2B	37	43	4F	5B	5B	5B	5B

KEY	ALT/ ⬆ / CTRL							
	000	001	010	011	100	101	110	111
←	88	88	88	88	88	88	88	88
TAB	89	89	89	89	89	89	89	89
↵	8D	8D	8D	8D	8D	8D	8D	8D
ESC	9B	9B	9B	9B	9B	9B	9B	9B
SPACE	AO	AO	AO	AO	AO	AO	AO	AO
' "	A7	A7	A2	A2	A7	A7	A2	A2
, <	AC	AC	BC	BC	AC	AC	BC	BC
- _	AD	AD	DF	9F	AD	AD	DF	9F
. >	AE	AE	BE	BE	AE	AE	BE	BE
/ ?	AF	AF	BF	BF	AF	AF	BF	BF
0)	BO	Bo	A9	A9	5C	5C	5C	5C
1 !	B1	B1	A1	A1	5D	5D	5D	5D
2 @	B2	80	CO	80	5E	5E	5E	5E
3 #	B3	B3	A3	A3	5F	5F	5F	5F
4 \$	B4	B4	A4	A4	60	60	60	60
5 %	B5	B5	A5	A5	7B	7B	7B	7B
6 ^	B6	9E	DE	9E	7C	7C	7C	7C
7 &	B7	B7	A6	A6	7D	7D	7D	7D
8 *	B8	B8	AA	AA	7E	7E	7E	7E
9 (	B9	B9	A8	A8	7F	7F	7F	7F
: :	BB	BB	BA	BA	BB	BB	BA	BA
= +	BD	BD	AB	AB	BD	BD	AB	AB
[{	DB	9B	FB	9B	DB	9B	FB	9B
\	DC	9C	FC	9C	DC	9C	FC	9C
DEL	FF	FF	FF	FF	FF	FF	FF	FF

KEY	ALT / ↑ /CTRL							
	000	001	010	011	100	101	110	111
] }	DD	9D	FD	9D	DD	9D	FD	9D
` ~	E0	E0	FE	FE	E0	E0	FE	FE
A	E1	81	C1	81	61	61	61	61
B	E2	82	C2	82	62	62	62	62
C	E3	83	C3	83	63	63	63	63
D	E4	84	C4	84	64	64	64	64
E	E5	85	C5	85	65	65	65	65
F	E6	86	C6	86	66	66	66	66
G	E7	87	C7	87	67	67	67	67
H	E8	88	C8	88	68	68	68	68
I	E9	89	C9	89	69	69	69	69
J	EA	8A	CA	8A	6A	6A	6A	6A
K	EB	8B	CB	8B	6B	6B	6B	6B
L	EC	8C	CC	8C	6C	6C	6C	6C
M	ED	8D	CD	8D	6D	6D	6D	6D
N	EE	8E	CE	8E	6E	6E	6E	6E
O	EF	8F	CF	8F	6F	6F	6F	6F
P	F0	90	D0	90	70	70	70	70
Q	F1	91	D1	91	71	71	71	71
R	F2	92	D2	92	72	72	72	72
S	F3	93	D3	93	73	73	73	73
T	F4	94	D4	94	74	74	74	74
U	F5	95	D5	95	75	75	75	75
V	F6	96	D6	96	76	76	76	76
W	F7	97	D7	97	77	77	77	77
X	F8	98	D8	98	78	78	78	78
Y	F9	99	D9	99	79	79	79	79
Z	FA	9A	DA	9A	7A	7A	7A	7A

5.1.2 Video Display Generator

The video display is the primary output device on the MPF-III. The MPF-III has the 80-column text display feature, in this case, a high-resolution video display is recommended.

Four modes of video displays are available on the MPF-III. They are:

- 1) Text 24 by 40 characters
- 2) Text 24 by 80 characters
- 3) Low-resolution graphics 40 by 48 blocks in 16 colors
- 4) High-resolution graphics 280 by 192 dots in 6 colors

In either text mode, all the ASCII characters including the upper/lowercase letters, numbers and special symbols can be displayed. In either graphics mode, four text lines (40 or 80 characters on each line) may be reserved at the bottom of the screen. In this case, they are called mixed modes.

In low-resolution graphics mode, the screen can display an array of 40 by 48 blocks in 16 colors. In mixed mode, the range is 40 by 40 due to the four lines of text display at the bottom of the screen.

In high-resolution graphics mode, the screen can display an array of 280 by 192 dots in six colors. In mixed mode, the range is 160 by 280 as a result of the four lines of text display at the bottom of the screen.

* Text mode

In text mode, all the upper- and lowercase letters, numbers and special symbols can be displayed. Each character uses a space 7 dots in width and 8 in height. The pattern of each character is formed by a 5 by 7 dot matrix. Adjacent characters on a same row are separated by a two-dot space while there is a one-row space between rows.

* Text Character Set

There are two character sets available on the MPF-III: the primary character set and the alternate character set. They are different only in display format. Consult the table on the next page.

Table 5-1 The ASCII Character Set

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	NUL	32	20	SP	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	\$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BEL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	HT	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SQ	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

There are three display formats on the MPF-III:

Normal: white on black

Inverse: black on white

Blinking: alternating between normal and inverse

Every character on the screen is stored in memory as a one-byte display data. The low order 6 bits constitutes the ASCII code of the character displayed while the high order 2 bits select inverse/flashing mode and upper/lower-case. In the primary character set, bit 7 selects inverse/normal mode while bit 6 controls the flashing of the character. In the alternate character set, bit 6 selects upper/lower-case but does not control the flashing of the character.

There are two text display modes on the MPF-III: 40-column text mode and 80-column text mode. The number of dots required to form the characters are the same, but the physical width of the characters in the 80-column text mode is half of that in the 40-column text mode.

* Low-resolution Graphics Mode

In low resolution graphics mode, the screen can display an array of 40 by 48 blocks, while each block may come in any of the 16 colors available on the MPF-III as shown in Table 5-2. But the blocks come in only white, black and gray on monochrome monitors. As there is no space preserved between blocks, adjacent blocks will melt into larger blocks.

The tint of colors may vary from one TV set to another. You may use the color tint controls for adjustment.

Decimal	Hex	Color	Decimal	Hex	Color
0	\$0	Black	8	\$8	Brown
1	\$1	Magenta	9	\$9	Orange
2	\$2	Dark blue	10	\$A	Gray 2
3	\$3	Purple	11	\$B	Pink
4	\$4	Dark Green	12	\$C	Light Green
5	\$5	Gray 1	13	\$D	Yellow
6	\$6	Medium Blue	14	\$E	Aquamarine
7	\$7	Light blue	15	\$F	White

Table 5-2 Low-resolution Graphics Colors

* High-resolution Graphics Mode

When your MPF-III operates in high resolution graphics mode, the screen displays an array of 53,760 dots (280 by 192). Each dot may come in one of the six colors--white, black, green, blue, orange and purple.

When operating in high resolution graphics mode, the MPF-III fetches screen information from a memory area of 8,192 bytes. This memory area is known as a display buffer. In fact there are two display buffers of the same size: page 1 and page 2. The memory range of page 1 or primary page is from \$2000 (hexadecimal) or 8192 (decimal) through \$3FFF or 16383, and the memory range of page 2 is from \$4000 through \$5FFF.

Each dot displayed on the screen (when the MPF-III operates in high resolution graphics mode) represents one bit in the screen buffer. The seven bits of a byte is displayed on the screen, while the remaining bit is used for selecting color for the seven bits displayed on the screen.

Each line on the screen requires 40 bytes of information. The first bit of the first byte is displayed as the leftmost bit (or at the leftmost position of a line), followed by the second bit, and third bit through the seventh bit. The eighth bit of the first byte is used for selecting color, it is not displayed. Following the seventh bit of the first byte is the first bit of the second byte, and the first bit of the third byte follows the seventh bit of the second byte. Therefore, each line consists of

$7 \text{ (dots)} * 40 \text{ (bytes)} = 280 \text{ (dots)}$

On a black-and-white TV or video monitor, if the corresponding bit of a dot is 1 or "on", the dot appears in white on the screen; if the corresponding bit is 0 or "off", the dot comes in black on the screen.

However, on a color TV or monitor, the color of a dot is determined not only by whether its corresponding bit is on or off, but also by the position where the dot appears on the screen. If the corresponding bit of the dot is off, the dot will come in black. But if the corresponding bit of the dot is on, the color of the dot will be decided by its position on a screen. If the dot appears on the column 0, leftmost column, or on even-numbered column it will appear violet. If the dot is in the right column (column 279) or an odd-numbered column, it will appear in green. If two dots come side by side, they will be in white. If the dot which is contained in a byte with the eight bit being one, then the purple and green colors will be replaced by blue and orange colors.

In high resolution graphics mode, there are six colors available. However, they are subject to the following limitations.

- 1) Dots on even-numbered columns only come in black, blue, or purple.
- 2) Dots on odd-numbered columns only come in black, green, or orange.
- 3) Each byte must come in either purple/green or blue/orange. It is absolutely impossible that purple/orange, purple/blue, green/orange, green/blue come in the same byte.
- 4) If two adjacent dots are both in color, they will be displayed in white, even if they belong to different byte.

* Display Page

The MPF-III generates the video displays by using the data stored in certain areas in memory. These areas are called display pages which are used as the buffers for displaying the data in your programs.

Every byte in the display buffer corresponds to a certain object on the screen. In text mode, the object is a character. In low-resolution graphics mode, it is a set of two blocks piled together. In high-resolution graphics mode, it is a row of seven contiguous dots.

In text/low-resolution graphics mode, two display pages (page 1 and page 2) of the same size are used. The ranges are \$400 through \$7FF and \$800 through \$BFF respectively.

In the 80-column text mode, twice the amount of data of video display is handled. The 80-column text mode uses two combination pages, each one composed of one page in the main memory and another in the auxiliary memory. A special soft switch is used to store data in these pages. In high-resolution graphics mode, each byte of memory is used to control a row of 7 dots wide by one dot high. As a result, eight times that of the memory space required in the text mode is necessary to store data for video display. Consult the table on the next page.

Display mode	Page	Lowest Address		Highest Address	
40-column Text, Low-resolution Graphics	1	\$400	1024	\$7FF	2047
	2	\$800	2048	\$BFF	3071
80-column Text	1	\$400	1024	\$7FF	2047
	2	\$800	2048	\$BFF	3071
High-resolution Graphics	1	\$2000	8192	\$3FFF	16383
	2	\$4000	16384	\$5FFF	24575

*Display Mode Switching

We can read or write to a special memory location (called a soft switch) to select display mode. On the MPF-III, most soft switches have three memory locations for turning the switch on, off or reading the current status of the switch. These special locations for the soft switches are listed in the following table.

To switch from the mixed mode to the full screen mode in a assembly program, we may use

```
STA $C052
```

In BASIC, you may use POKE 49234,0 instead.

On the MPF-III, memory locations from \$C000 through \$C01F are preserved for handling keyboard data and strobe; you can read it to get data or write it to clear strobe. When you write to a soft switch, it does not matter what value you write.

Name	Function	Location		Notes
		Hex	Decimal	
ALT-CHARSET	Alternate char. set on	\$C00F	49167	W
	Alternate char. set off	\$C00E	49168	W
	Read ALTCHARSET switch	\$C01E	49182	R

TEXT	Text mode on	\$C051	49233	
	Text mode off (graphics)	\$C050	49232	
	Read TEXT switch	\$C01A	49178	R
MIXED	Mixed-mode on	\$C053	49235	1
	Mixed-mode off	\$C052	49234	1
	Read MIXED switch	\$C01B	49179	R
PAGE2	Page 2 on	\$C055	49237	2
	Page 2 off (Page 1)	\$C054	49236	2
	Read PAGE2 switch	\$C01C	49180	R
HIRES	Hi-res mode on	\$C057	49239	1
	Hi-res mode off	\$C056	49238	1
	Read HIRES switch	\$C01D	49181	R
80COL	80-column display on	\$C00D	49165	W
	80-column display off	\$C00C	49164	W
	Read 80COL switch	\$C01F	49183	R
80STORE	Store in auxiliary memory	\$C001	49153	W,3
	Store in main memory	\$C000	49152	W,3
	Read 80STORE switch	\$C018	49176	R
VBL	Read vertical blanking	\$C019	49177	R,4
AUXPG	Auxiliary display page 1	\$C07A	49274	W
	Auxiliary display page 2	\$C07B	49275	W
	Read AUXPG switch	\$C1C3	48603	R
AKD	Read AKD status, clear keyboard strobe signal	\$C010	49168	R
BANK1	Read BANK1 switch	\$C011	49169	R
RDRAM	Read RDRAM switch	\$C012	49170	R

Table 5-3 Video Display Page Locations

Note:

- 1) This mode is effective only when the graphics-mode switch is on.
- 2) When the 80-column auxiliary text page is enabled for writing, this switch has a different function.
- 3) This switch changes the function of PAGE2 switch for addressing the primary text memory on the extended 80-column card.
- 4) Reading this location returns the status of the vertical blanking signal VBL.

* Addressing Display Pages Directly

The display memory maps are shown in Fig. 5-1, 5-2, 5-3 and 5-4. All of the different display modes use the same basic addressing scheme: characters or graphics bytes are stored as rows of 40 contiguous bytes, but the rows themselves are not stored at locations corresponding to their locations on the display. Instead, the display address is transformed so that three rows that are eight rows apart on the display are grouped together and stored in the first 120 locations of each block of 128 bytes. By folding the display data into memory this way, the MPF-III stores all 960 characters of displayed text within 1K bytes of memory.

The high-resolution graphics display is stored in much the same way as text, but there are eight times as many bytes to store because eight rows of dots occupy the same space on the display as one row of characters. The subset consisting of all the first rows from the groups of eight is stored in the first 1024 bytes of the high-resolution display page. The subset consisting of all the second rows from the groups of eight is stored in the second 1024 bytes, and so on for a total of 8 times 1024, or 8192 bytes. In other words, each block of 1024 bytes in the high-resolution display page contains one row of dots out of every group of eight rows. The individual rows are stored in sets of three forty-byte rows, the same way as the text display.

All of the display modes can use either of two display pages. The display maps show addresses only for each Page 1. To obtain addresses for text or low-resolution graphics Page 2, add 1024(\$400); to obtain addresses for high-resolution Page 2, add 8192(\$2000).

The 80-column display works a little differently. Half of the data is stored in the main memory, and the other half is stored in the auxiliary memory using the same addresses. The display circuitry fetches bytes from these two memory areas simultaneously and displays them sequentially; first the byte from the auxiliary memory, then the byte from the main memory. The main memory stores the characters in the odd columns of the display, and the auxiliary memory stores the characters in the even columns.

To store display data on the auxiliary memory, first turn on the 80STORE soft switch by writing to location 49153 (hexadecimal \$C001). With 80STORE on, the page-select switch PAGE2 selects between the portion of the 80-column display stored in main memory and the portion stored in the auxiliary memory. To select the 80-column text, turn the PAGE2 soft switch on by reading or writing at location 49237.

		0 \$00	1 \$01	2 \$02	3 \$03	4 \$04	5 \$05	6 \$06	7 \$07	8 \$08	9 \$09	10 \$0A	11 \$0B	12 \$0C	13 \$0D	14 \$0E	15 \$0F	16 \$10	17 \$11	18 \$12	19 \$13	20 \$14	21 \$15	22 \$16	23 \$17	24 \$18	25 \$19	26 \$1A	27 \$1B	28 \$1C	29 \$1D	30 \$1E	31 \$1F	32 \$20	33 \$21	34 \$22	35 \$23	36 \$24	37 \$25	38 \$26	39 \$27			
\$400	1024																																											
\$480	1152																																											
\$500	1280																																											
\$580	1408																																											
\$600	1536																																											
\$680	1664																																											
\$700	1792																																											
\$780	1920																																											
\$428	1064																																											
\$4A8	1192																																											
\$528	1320																																											
\$5A8	1448																																											
\$628	1576																																											
\$6A8	1704																																											
\$728	1832																																											
\$7A8	1960																																											
\$450	1104																																											
\$4D0	1232																																											
\$550	1360																																											
\$5D0	1488																																											
\$650	1616																																											
\$6D0	1744																																											
\$750	1872																																											
\$7D0	2000																																											

Fig. 5-1 Map of 40-column Text Display

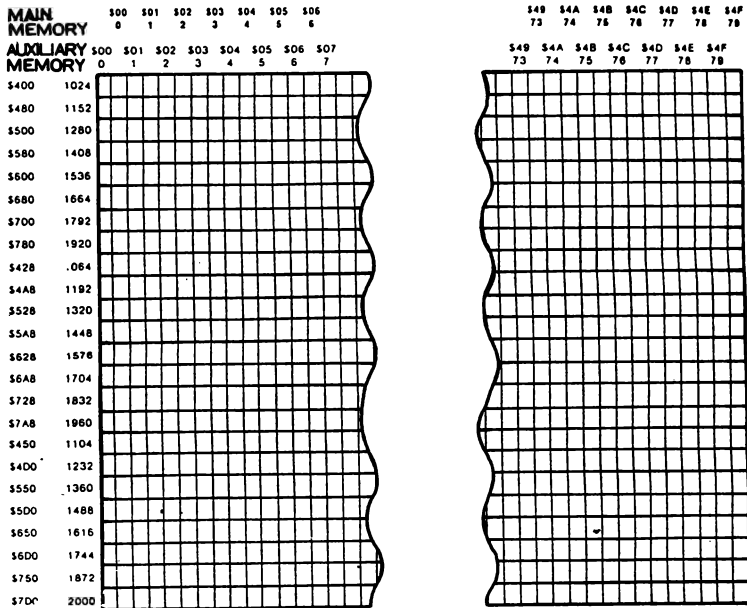


Fig. 5-2 Map of 80-column Text Display

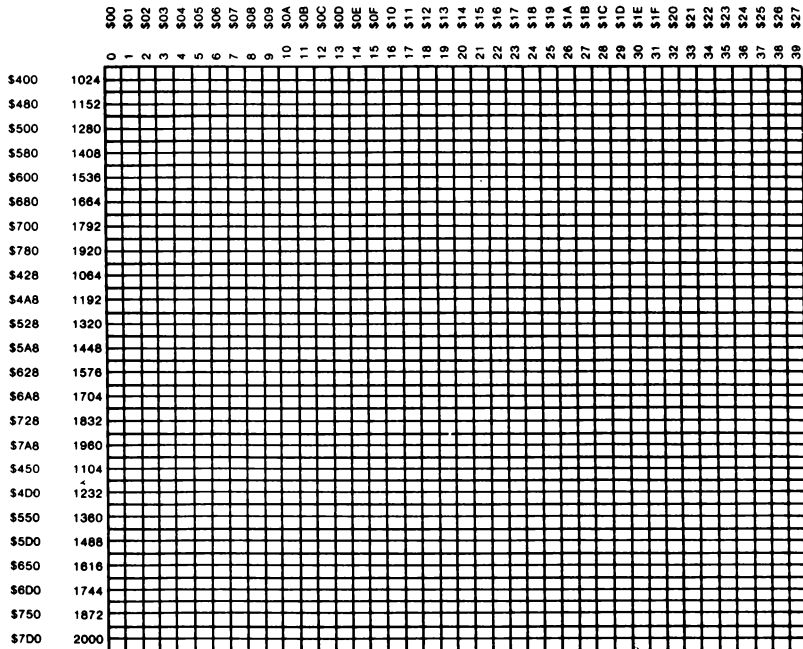


Fig. 5-3 Map of Low-resolution Graphics Display

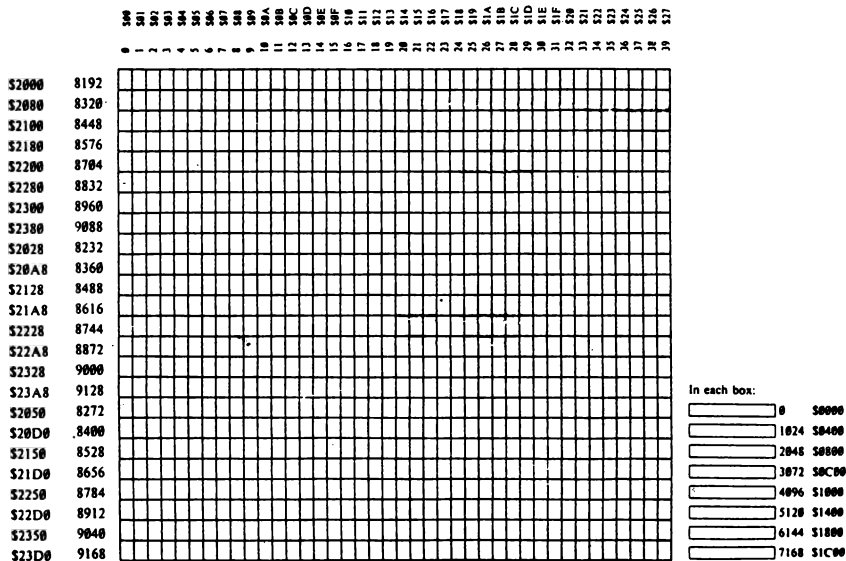


Fig. 5-4 Map of High-resolution Graphics Display

5.2 Secondary Inputs and Outputs

In addition to primary I/O devices (keyboard and display), there are the secondary inputs/outputs such as speaker output, cassette I/O, annunciator output, strobe output, switch input, analog input, hand control connector signals, printer interface output, and sound generator. We may control them by reading or writing some special memory locations.

5.2.1 Speaker

The MPF-III has a built-in speaker controlled by soft switch, we can make the speaker send out sounds by reading the specified memory location. The switch location of the speaker is 49200 (\$C030).

5.2.2 Cassette I/O

On the back panel of the MPF-III you can find two earphone jacks which are used for cassette I/O. Through them we read can data from the magnetic tape into the

MPF-III, or store data onto the tapes. The cassette output switch is located at 49184 (\$C020), and the cassette input switch is located at 49248 (\$C060). Each time you read these positions you can get a byte, but only bit 7 is significant. In a machine language program, bit 7 is used as a sign bit. Therefore, it is possible to execute a Branch plus or Branch minus upon reading this byte. In BASIC, we can use READ and WRITE to perform I/O.

5.2.3 Hand Control Connector Signals

On the right side of the MPF-III you can find a 9-pin D-type connector which has three 1-bit inputs as well as four analog inputs. You can connect devices such as a joystick through them.

5.2.4 Annunciator Output

The MPF-III has four 1-bit outputs called annunciators which can be used to control signal lamps, relay or the on/off of other electronic devices. Annunciators are controlled by soft switches at dedicated positions. Consult the table on the next page.

No.	Annunciator Pin	State	Address	
			Decimal	Hex
0	15	off	49240	\$C058
		on	49241	\$C059
1	14	off	49242	\$C05A
		on	49243	\$C05B
2	13	off	49244	\$C05C
		on	49245	\$C05D
3	12	off	49246	\$C05E
		on	49247	\$C05F

Table 5-4 Annunciator Memory Locations

Note: Pin numbers are given for the 16-pin IC connector on the circuit board.

5.2.5 Strobe Output

The strobe signal is usually used to control the data latching of the external devices. Customarily we write when we use it. However, you may read to activate the strobe if singal pulse alone is required. The memory location for strobe signal is 49216 (\$C040).

5.2.6 Switch Input

The three 1-bit inputs may be connected to electronic devices or pushbuttons for use. When you read a byte from one of these locations, only bit 7 is significant. As a result, you may use them in combination with a Branch plus or Branch minus in a machine language program. In BASIC, you may PEEK to read the switch, and compare the value with 128; if it is equal to or greater than 128, the switch is on. The memory locations are 49249 through 49251 (\$C061 through \$C063).

Function	Address		Notes
	Decimal	Hex	
Speaker	49200	\$C030	Read
Cassette out	49184	\$C020	Read
Cassette in	49248	\$C060	Read
Annunciator 1 on	49241	\$C059	
Annunciator 2 off	49240	\$C058	
Annunciator 1 on	49243	\$C05B	
Annunciator 2 off	49242	\$C05A	
Annunciator 1 on	49245	\$C05D	
Annunciator 2 off	49244	\$C05C	
Annunciator 1 on	49247	\$C05F	
Annunciator 2 off	49246	\$C05E	
Strobe Output	49216	\$C040	Read
Analog Input Reset	49264	\$C070	
Analog Input 0	49252	\$C064	Read
Analog Input 1	49253	\$C065	Read
Analog Input 2	49254	\$C066	Read
Analog Input 3	49255	\$C067	Read

Table 5-5 Secondary I/O Memory Locations

5.2.7 Analog Input

A timing circuit is formed with four analog inputs together with a 150 k ohm variable resistor. In a program, you have to reset the timing circuitry before you read the analog inputs. You can reset the timing circuitry by accessing location 49264 (\$C070). Once the timing circuitry is reset, bit 7 of the bytes at locations 49252 through 49255 (\$C064 through \$C067) will become 1. If you use PEEK in BASIC, you will get a value greater than 128. Within 3 ms, these bits will become 0 and they will remain 0 until you reset them again. The exact time that these four bits remain high will be proportional to the resistance they are connected to. To read the analog inputs in a machine language program, you may use a loop to reset the timer and then increment the counter until the bits at the appropriate locations become 0. Another way to do this is to use the built-in subroutine PDLRED.

5.2.8 Printer Interface

The MPF-III provides a printer driver to select the type of printers through a switch or software. You may read location \$C026 to select EPSON, C.Itoh or CP-80 printer. When it is ON, the value is less than 128; otherwise it is equal to or greater than 128. Consult the following table:

Function	Loacation	R/W	Remark
Read printer busy status	\$C1C1	R	Busy: >=128
Read printer ACK status	\$C1C0	R	
Write printer data	\$C090-\$C093	W	
Read EPSON/C.Itoh switch	\$C026	R	On : <128 Off: >=128

Table 5-6 Printer Interface Memroy Locations

5.2.9 Sound Generator

By reading the specific locations shown in the following table together with a number of subroutines, you can produce the sounds of four musical instruments (piano, bell, organ and xylophone) and a variety of special sound effects simulating the car, the train, the airplane, the laser gun and explosion.

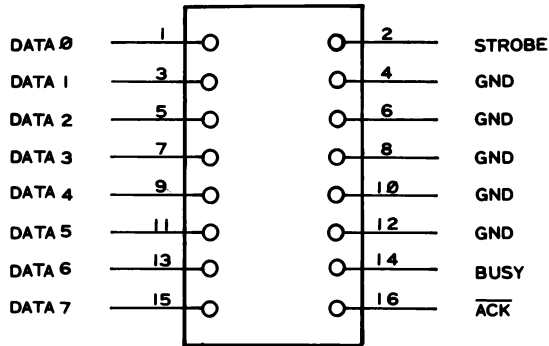
Function	Location	R/W
Disable sound generator	\$C074	
Read sound generator	\$C075	Read
Write to sound generator	\$C076	Write
Lock the addresses in the sound generator	\$C077	Write

Table 5-7 Sound Generator Memory Locations

5.3 Peripheral Connectors

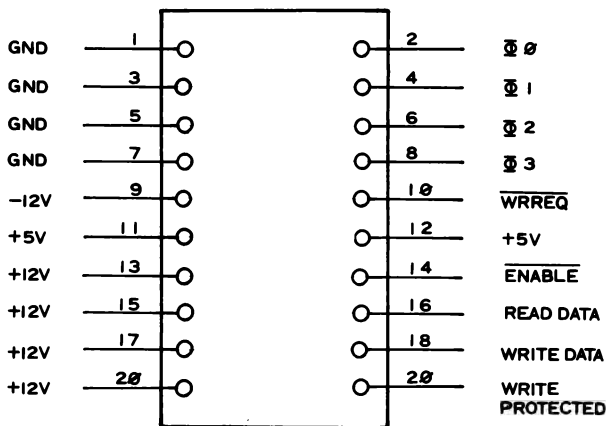
5.3.1 Printer Connector

You can find the 16-pin printer connector on the back panel of the MPF-III system unit. See the following figure:



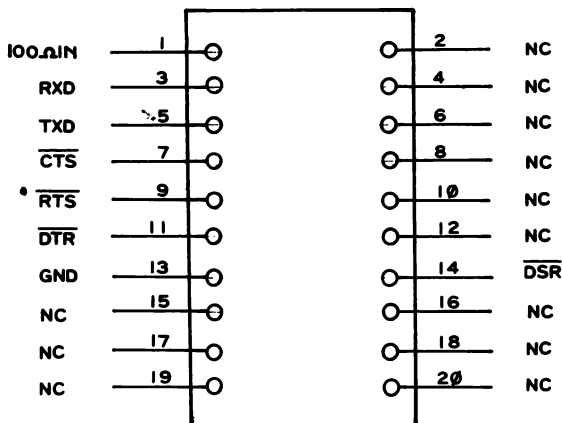
5.3.2 Disk Drive Connector

To use a disk drive, you have to insert the FDI into the appropriate slot inside the system unit and insert the 20-pin connector into the socket marked "DRIVE 1" or "DRIVE 2" on the back panel. Consult the following pin configuration:



5.3.3 RS-232 Interface Connector

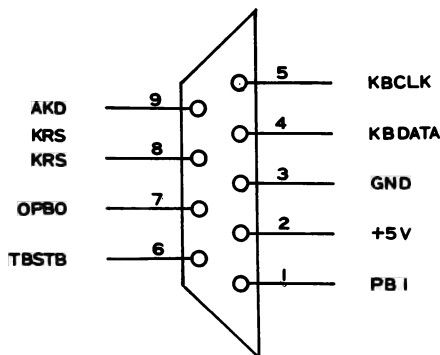
To use the RS 232 interface, you have to insert one end of the RS 232 interface cartridge into the cartridge socket on the right of the system unit, and connect the other end to external devices. Consult the following pin configuration.



5.3.4 Keyboard Connector

The MPF-III keyboard is connected to the system unit through a 9-pin ribbon cable. The pin configuration of the connector is given below.

PB0 and PB1 handle the signals read into the system unit. AKD stands for "any key down", the signal is high if any key is pressed, otherwise it remains low.



5.3.5 Joystick Connector

On the right of the system unit you can find a 9-pin connector marked "JOYSTICK" as shown in Fig.A. Another 16-pin connector is provided at the lower right corner of the main circuit board as shown in Fig.B.

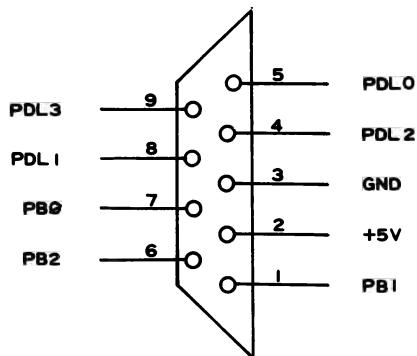


Fig.A

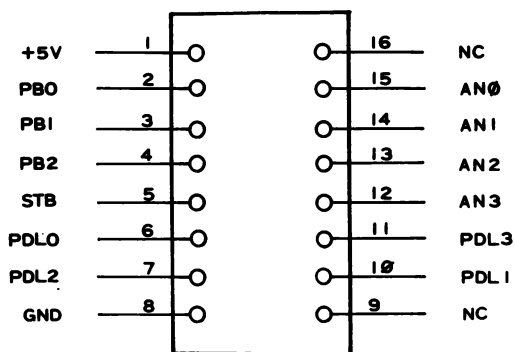


Fig.B

5.4 Built-in I/O Firmware

Almost every program on the MPF-III takes input from the keyboard and sends output to the display. The Monitor and the BASIC do this by means of standard I/O subroutines that are built into the MPF-III firmware. Many applications programs also use the standard I/O subroutines.

This chapter describes the features of these subroutines as they are used by the Monitor and by the BASIC interpreter, and tells you how to use the standard subroutines in your assembly language programs.

Subroutine Name	Location	Description
CONOUT	\$FDED	Console out: sends a character to the display.
CONIN	\$FD0C	Console in: displays the blinking cursor; goes to the standard input routine, normally KBDIN.
KBDIN	\$FD1B	Keyboard in : Accepts a character from the keyboard.
RDLINE	\$FD6A	Read line: displays the prompt character; accepts a sequence of characters by means of CONIN.

Table 5-8 Standard I/O Subroutines

* The Standard I/O Links

When you call one of the character I/O subroutines (CONOUT and CONIN), the first thing that happens is an indirect jump to an address stored in programmable memory. Memory locations used for transferring control to other subroutines are called I/O links.

By calling the I/O subroutines that jump to the link addresses instead of calling the standard subroutines directly, you ensure that your program will work properly in conjunction with other software, such as DOS or a printer driver, that changes one or both of the I/O links. For the purposes of this chapter, we shall assume that the I/O links contain the addresses of the standard I/O subroutines CONOT 1 and KBDIN.

5.4.1 Standard Output Features

The standard output routine is named CONOUT. CONOUT normally calls CONOT 1 which sends one character to the display, advances the cursor position, and scrolls the display when necessary. In this case, the cursor is placed at the lower left corner. CONOT 1 restricts its use of the display to an active area called the text window, described below.

* CONOUT Output Subroutine

Your program makes a subroutine call to \$FDED with a character in the accumulator. CONOUT then passes control via the output link CSW (character output switch) to the current output, normally CONOT 1, which takes the character in the accumulator and writes it out. If the accumulator contains a control character, CONOT 1 either performs one of the special functions described below or ignores the character.

Control Character	ASCII Name	Action Taken by CONOT 1
CTRL-G	(BEL)	Produces a 1000Hz tone for 0.1 seconds.
CTRL-H	(BS)	Backspace Moves cursor position one space to the left.
CTRL-J	(LF)	Moves cursor position down to next line in window; scrolls if cursor is on the bottom of window.
CTRL-K	(VT)	Clears from cursor position to the end of the window.

CTRL-L	(FF)	Moves cursor position to upper-left corner of widow and clears window.
CTRL-M	(CR)	Moves cursor position to left end of next line in window; scrolls if needed.
CTRL-N	(SO)	Sets display format normal.
CTRL-O	(SI)	Sets display format inverse.
CTRL-Q	(DC1)	Sets display to 40-column.
CTRL-R	(DC2)	Sets display to 80-column.
CTRL-S	(DS3)	Stops sending characters to the display, until a key is pressed.
CTRL-U	(NAK)	Deactivates 80-column firmware, homes cursor, and clears screen.
CTRL-V	(SYN)	Scrolls the display down one line, leaving the cursor in the current position.
CTRL-W	(ETB)	Scrolls the display up one line leaving the cursor in the current position.
CTRL-Y	(EM)	Moves cursor position to upper-left corner of window (but doesn't clear).
CTRL-Z	(SUB)	Clears the line the cursor position is on.
CTRL-\	(FS)	Moves cursor position one space to the right.
CTRL-]	(GS)	Clears line from cursor position to the right end of the window.
CTRL-^	(RS)	Using the next two characters, minus 32, as one-byte X and Y values, moves the cursor position to CH=X, CV=Y.

Table 5-9 Control Characters with CONOT1

CONOT 1 does not display control characters. Instead, the control characters listed in the preceding table are used to initiate some actions by the firmware. Most of the functions listed here can also be invoked from the keyboard.

*The Stop-list Feature

When you are using any program that display text via CONOT 1 you can make it stop by holding down the CTRL key and pressing the S key. Whenever CONOT 1 receives a carriage return from the program, it checks to see if you have typed a CTRL-S. If you have, CONOT 1 stops and waits for you to press another key. When you want CONOT 1 to resume, press another key; CONOT 1 will send the carriage return it got earlier to the display, then continue normally. The character code of key you pressed to resume displaying is ignored unless it is a CTRL-C. CONOT 1 passes CTRL-C back to the program; if it is a BASIC program, this enables you to terminate the program while in stop-list mode.

The Text Window

After starting up the computer or after a reset, CONOT1 uses the entire display. However, you can restrict CONOT 1's activity to any rectangular portion of the display you wish. The active portion of the display is called the text window. CONOT 1 puts characters only into the window; when it reaches the end of the last line in the window, it scrolls only the contents of the window.

You can set the top, bottom, left side, and width of the text window by storing the appropriate values into four locations in memory. This enables your programs to control the placement of text in the display and to protect other portions of the screen from being written over by new text.

Memory location 32 (hexadecimal \$20) contains the number of the leftmost column in the text window. This number is normally 0, the number of the leftmost column in the display. In a 40-column display, the maximum value for this number is 39 (hexadecimal \$27); in an 80-column display, the maximum value is 79 (hexadecimal \$4F).

Memory location 33 (hexadecimal \$21) holds the width of the text window. For a 40-column display, it is normally 40 (hexadecimal \$28); for an 80-column display, it is normally 80 (hexadecimal \$50). CONOT 1 truncates the width to an even value.

Note: Be careful not to let the sum of the window width and the leftmost position in the window exceed the width of the display you are using (40 or 80). If this happens, it is possible for CONOT 1 to put characters into memory locations outside the display page, possibly destroying programs or data.

Memory location 34 (hexadecimal \$22) contains the number of the top line of the text window. This is normally 0, the topmost line in the display. Its maximum value is 23 (hexadecimal \$17).

Memory location 35 (hexadecimal \$23) contains the number of the bottom line of the screen, plus 1. It is normally 24 (hexadecimal \$18) for the bottom line of the display. Its minimum value is 1.

Note: Any time you change the boundaries of the text window, you should make sure that the current cursor position (stored at CH and CV) is inside the new window. If it is outside, it is possible for CONOT 1 to put characters into memory locations outside the display page, possibly destroying programs or data.

* Inverse and Flashing Text

Subroutine CONOT 1 can display text in normal format, inverse format, or, with some restrictions, flashing format. The display format for any character in the display depends on two things: the character set being used at the moment, and the setting of the two high-order bits of the character's byte in the display memory.

As it sends your text characters to the display, CONOT1 sets the high-order bits according to the value stored at memory location 50 (hexadecimal \$32). If that value is 255 (hexadecimal \$FF), CONOT 1 sets the characters to display in normal format; if the value is 63 (hexadecimal \$3F), CONOT 1 sets the characters to inverse format. If the value is 127 (hexadecimal \$7F) and if you have selected the primary character set, the characters will be displayed in flashing format. Note that flashing format is not available in the alternate character set.

To control the display format of the characters, routine CONOT 1 uses the value at location 50 as a logical mask to force the setting of the two high-order bits of each character byte it puts into the display page. It does this by performing the logical AND function on the data byte and the mask byte. The result byte contains a 0 in any bit that was 0 in the mask.

If you set the mask value at location 50 to 127 (hexadecimal \$7F), the high-order bit of each result bytes will be 0, and the characters will be displayed either as lowercase or as flashing, depending on which character set you have selected. In the primary character set, the next-highest bit, bit 6, selects flashing format with uppercase characters. With the primary character set you can display lowercase characters in normal format and uppercase characters in normal, inverse, and flashing formats. In the alternate character set, bit 6 selects lowercase or special characters. With the alternate character set you can display uppercase and lowercase characters in normal and inverse formats.

5.4.2 Standard Input Features

The MPF-III's firmware includes two different subroutines for reading from the keyboard. One subroutine is named CONIN which stands for console in. It calls the standard character input subroutine KBDIN which accepts one character at a time from the keyboard. The other subroutine is named RDLIN, which stands for read line. By making repeated calls to CONIN RDLIN accepts a sequence of characters terminated with a carriage return.

* CONIN Input Subroutine

A program gets a character from the keyboard by making a subroutine call to CONIN at memory location \$FD0C. CONIN sets the character at the cursor position to flash, then passes control via the input link KSW to the current input subroutine, which is normally KBDIN.

CONIN displays a cursor at the current cursor position, which is immediately to the right of whatever character you last sent to the display.

* KBDIN Input Subroutine

KBDIN is the standard input subroutine. When called, it waits until the user presses a key, then returns with the key code in the accumulator.

The problem of displaying a cursor without using flashing format is handled by KBDIN. If the firmware is active, KBDIN displays a steady inverse space (rectangle) , unless you are in escape mode, when it displays a plus sign(+) in inverse format

KBDIN also generates a random number. While it is waiting for the user to press a key, KBDIN repeatedly increments the 16-bit number in memory locations 78 and 79 (hexadecimal \$4E and \$4F). This number keeps increasing from 0 to 65535, then starts over again at 0. The value of this number changes so rapidly that there is no way to predict what it will be after a key is pressed. A program that reads from the keyboard can use this value as a random number or as a seed for a pseudo-random number routine.

Escape Codes with KBDIN

KBDIN has many special functions that you invoke by typing escape codes on the keyboard. An escape code is obtained by pressing the ESC key, releasing it, and then pressing some other key, as shown in the table on the next page.

Escape Code	Function
ESC @	Clears the window and homes the cursor
ESC A	Moves the cursor up one line
ESC B	Moves the cursor right one space
ESC C	Moves the cursor left one space
ESC D	Moves the cursor down one line
ESC E	Clears to the end of the line
ESC F	Clears to the bottom of the window
ESC I	Moves the cursor up one line and turns
ESC ↑	on escape mode
ESC J	Moves the cursor left one space and
ESC ←	turns on escape mode
ESC K	Moves the cursor right one space and
ESC →	turns on escape mode
ESC M	Moves the cursor down one line and turns
ESC ↓	on escape mode
ESC R	Turns on restricted-case mode
ESC T	Turns off restricted-case mode
ESC 4	Switches to 40-column mode, homes the cursor, and clears the screen
ESC 8	Switches to 80-column mode, homes the cursor, and clears the screen
ESC CTRL-Q	Deactivates the 80-column firmware

Table 5-10 Escape Codes

* Cursor Motion in Escape Mode

In escape mode, you can keep using the cursor-motion keys without pressing the ESC key again. This enables you to perform repeated cursor moves by holding down the appropriate key.

RDLINE Input Subroutine

Programs often need strings of characters as input. While it is possible to call CONIN repeatedly to get several characters from the keyboard, there is a more powerful subroutine you can use. This routine is named RDLINE which stands for read line, and it starts at location \$FD6A. Using repeated calls to CONIN, RDLINE accepts characters from the standard input subroutine - usually KBDIN and puts them into the input buffer located in the memory page from \$200 to \$2FF.

The first thing RDLINE does when you call it is display a prompting character, called simply a prompt. The prompt indicates to the user that the program is waiting for input. Different programs use different prompt characters, helping to remind the user which program is requesting the input.

RDLINE uses the character stored at memory location 51 (hexadecimal \$33) as the prompt character.

As the user types the character string, RDLINE sends each character to the standard output routine - normally CONOUT 1-which displays it at the previous cursor position and puts the cursor at the next available position on the display, usually immediately to the right. As the cursor travels across the display, it indicates the position where the next character will be displayed.

RDLINE stores the characters in its buffer, starting at memory location in \$200 and using the X register to index the buffer. RDLINE continues to accept and display characters until the user presses the RETURN key; then it clears the remainder of the line the cursor is on, stores the carriage-return code in the buffer, sends the carriage-return code to the display, and returns to the calling program.

The maximum line-length that RDLINE can handle is 255 characters. If the user types more than this, RDLINE sends a backslash (\) and a carriage return to the display, cancels the line it has accepted so far, and starts over. To warn the user that the line is getting full, RDLINE sounds a bell (tone) at every keypress after the 248th.

* Editing with RDLINE

Subroutine RDLINE provides the standard on-screen editing features used by the BASIC interpreters and the Monitor.

* Cancel Line

Any time you are typing a line, pressing CTRL-X causes RDLINE to cancel the line. RDLINE displays a backslash (\) and issues a carriage return, then display the prompt and waits for you to type a new line. RDLINE takes the same action when you type more than 255 characters, as described above.

* Backspace

When you press the  key, RDLINE moves its buffer pointer back one space, effectively deleting the last character in its buffer. It also sends a backspace character to routine CONOUT, which moves the display position and the cursor back one space. If you type another character now, it will replace the character you backspaced over, both on the display and in the line buffer.

* Retype

The  key has a function that is complementary to the backspace function. When you press the  key, RDLINE picks up the character at the display position just as if it had been typed on the keyboard. You can use this procedure to pick up characters that you have just deleted by backspacing across them. You can use the backspace and retype functions with the cursor motion functions to edit data on the display.

Appendix A

Glossary

6502: The name of the microprocessor which is the brain of the MPF-III. Manufacturers of this microprocessor include Rockwell, Synertek, MOS Technology.

Address: When used as a noun, it is the specific value given to a memory location. The memory of the MPF-III ranges from 0 to 65535. As a verb, to refer to a specific memory location.

Addressing Mode: The methods with which a microprocessor refers to memory locations. The 6502 has thirteen addressing modes. Thus, it has thirteen ways of referring to memory.

Analog: Using continuously variable numbers to represent physical quantities such as voltage, length, resistance, etc. (Contrasted with digital).

AND: A logical operator or binary function that has the property that if P (an input) statement and Q (another input) statement are both true, then P AND Q are true (positive or on), false (negative or off) if either is false or both are false.

Application program: Written for specific purpose or tasks. It includes word processing, data management, graphics and communications. (Contrasted with system program).

Application software: The software system that is composed of application programs.

ASCII: American Standard Code for Information Interchange. A standard 8-bit information coding system that assigns a unique value from 0 to 127 to each of 128 characters, numbers, special characters, and control characters. It is used widely with most computers and terminals and can be transmitted parallel or serial.

Assembler: A program used to convert a source assembly language program, including mnemonics and symbols, to an object code (in binary representation) that can be recognized by the computers.

Assembly language: The structure of this programming language is similar to that of machine code language. Programs written in assembly language consist of mnemonics, symbols, and values. It is much more efficient to program in assembly language than to program in other programming languages.

Back Panel: When users face a machine, the back panel is on the opposite side and invisible. There are switch and jacks of electricity, peripheral equipment, video monitor and TV on MPF-III's back panel.

BASIC: Beginner's All-purpose Symbolic Instruction Code. Developed at Dartmouth College by Kemeny and Kurtz in 1963, it is probably one of the easiest programming languages to learn and master, and is probably the most widely used language for home and personal computers.

Binary: A number system with only two digits "0" and "1". Inside all computers, binary arithmetic operations are used for quick computing.

Binary function: An operation performed by an electronic circuit which has one or more inputs and only one output. All inputs and outputs are binary signals.

Bit: The most basic unit of data that can be recognized and processed by all computers. A bit is either 0 or 1. Bits can be grouped to form larger unit of information -- nibble, byte.

Board: See printed circuit board.

Bootstrap (boot): To start a system from a "cold" reset cycle. This term was derived from the sense that the computer intends to get itself started by pulling its bootstraps.

Buffer: An area in the memory in which information is to be stored temporarily, and will be output or processed later.

Bug: An error. A hardware bug is an electrical, mechanical, or electronic defect that interferes with the normal operation of a computer. A software bug is a mistake in programming.

Bus: A group of wires used to carry a set of related signals or information from one place to another.

Byte: A basic unit of information that is stored or processed by the computer. In some computers, a byte consists of seven bits. But in most of today's computers, a byte is made up of eight bits. As a byte consists of eight bits, one byte can represent or hold a value of from 0 up to 255. Each character in the ASCII character set can be represented by one byte. Each and every memory location of the MPF-III is one 8-bit byte.

Call: As a verb, to leave the program or the subroutine which is now being executed, jump to another program and execute that program, and then return to the original program. As a noun, an instruction that calls a subroutine or program.

Character: One symbol of a set of basic symbols such as those corresponding to the keys on a standard typewriter keyboard. The symbols usually include the decimal letters from 0 to 9, the letters from A through Z, punctuation marks, operation marks, operation symbols, or other special signs the computer can read, write, and process.

Chip: The equivalent of integrated circuit -- tiny silicon chip on which electronic circuits are implemented on a thin metal oxide film.

Code: A system of symbols for representing data or instructions.

Cold-start: To begin to operate computer which has just been turned on.

Color burst: The signal that color TV sets recognize and convert to the color dots which are then displayed on the TV screen. Without the color burst signal, all pictures would be black and white.

Command: The information that users key into a computer which then executes the assigned task.

Compiler: A language program; used to convert programs written in high-level languages to low-level languages which the computer recognize to facilitate execution.

Computer: Any device that can receive and store a set of instructions and information and process them in a predetermined and predictable way.

Computer aided-instruction (CAI): To help learners study by way of computers.

Connector: Jacks used to connect hardware sets of a computer system.

Control character: Characters in ASCII character set which have no graphic representation, and thus can not be seen on the screen. These characters are used for various control functions.

Copy-protect: Preventing data stored from copying. Diskettes for sale usually needs this process.

CRT: Cathod ray tube. Usually used to describe any device that has a TV screen.

Cursor: A special symbol on a screen, showing the position where a character can be typed in.

Daisy-wheel printer: A printer which types characters or special symbols by tipping a piece of daisy-like printing mode.

Data: Information of any type.

Data base: An organized data base sorts data by way of computer processing.

Data base management system: A software system used to organize, store, and modify data in a data base.

Debug: Find and correct errors.

Device: 1) An instrument or equipment used for a specific purpose or work. 2) A set of computer system.

DIP: Dual in line package. It is the most popular package method for integrated circuits. It has two parallel rows of pins. The numbers of pins usually come in 14, 16, 18, 20, 24, 40.

Disassembler: A program used to convert object machine code programs to assembly language programs.

Disk: A round plastic sheet used to store data. The data is stored as dots-like magnetized points. Its principle is like that of storing sound on a magnetized tape.

Disk controller card: A peripheral equipment card used to connect one or two disk drives to MPF-III and control the operation.

Disk drive: A peripheral equipment reads and inputs data from disks.

Diskette: Refer to a 5-1/4 inch small soft disks.

Disk envelope: It is used to keep disks, which should be removed when put a disk into a disk drive.

Disk Jacket: It is a permanent apparatus to protect disks which is made of black paper or plastics.

Disk operating system: A selective software system that enables computer to control and operate more than one disk drives.

Display: As a noun, denotes any devices that displays. As a verb, to show data on the screen.

Display device: An apparatus to make data visible. TV receiver and screen monitor are two examples.

Dot matrix printer: Characters printed by this machine are composed of dots. Each character occupies equal space.

Edge connector: A connector to which the edge of a printed circuit board can plug for electronic signal exchange.

Edit: To change or modify words in a file.

Entry point: Specific points or places in a subroutine where control can be transferred and re-entered. The entry point usually corresponds to a new or different function to be performed.

Error message: It is displayed when errors occurred.

Escape sequence: A series of key-in sequence, starting with ESC, is used to place the cursor and control characters displayed on the screen.

Exclusive OR: A binary function whose result is "off" or "false" when both inputs are "on" (true) or "off" (false); "on" when only one input is "on" (true).

Execute: To do a specific or a series of actions instructed by a program.

Feature: As a noun, a modification used in a sales department.

Firmware: Computer programs installed in an apparatus, usually EPROM or ROM. These programs (as MPF-III's monitor program) may execute at any time, but not be modified or deleted.

File: A collective object for data. It is usually assigned a name and stored in a peripheral storage media (as a disk).

Filename: The name for a file.

Flexible disk: The disk that is made of flexible plastics.

Format: A predetermined way to arrange characters, lines, punctuations, lines, etc. As a verb, to specify the form of a format.

Fortran: A widely used high level language, especially apply to number calculation, such as mathematics, engineering and science.

Graphic: Visible as a distinct, recognizable shape or color.

Graphics: A capability or a system to display graphic items.

Hand control: A selective peripheral installation to connect MPF-III's hand control connector. A joystick and a push are used to control operation of game program and other aspects.

Hand control connector: A nine-pin connector on MPF-III's right side panel, which is used to connect the hand control to the computer.

Hardware: The physical components of a computer.

HC: Hard copy. As a verb, to print what is on the screen onto paper, using a printer.

Hexadecimal: A number system in which the decimal 0 through 9 are used to represent the hexadecimal 0 through 9, and the decimal values 10 through 15 are represented by the letters from A to F. The hexadecimal values in this manual begin with a \$.

High level language: The programming languages that are close to human languages.

High order: The most important, or item which contains the highest value, of a set of similar items. The high order bit is the leftest or the most significant bit.

High part: The high order byte of a two-byte address. In decimal, the high part of an address is the quotient of the address divided by 256. In 6502 and many other microprocessors, the high part of an address comes last when stored in memory.

High-resolution graphics: Graphics displayed on MPF-III screen, including 280 by 192 dots in six colors.

Hz: Hertz, a measurement unit of frequency, cycle per second. 6502 operates at 1,023,000Hz.

Initialize: 1) Setting up a beginning status in preparation of operation, as to assign an initial value for operation byte's address. 2) A process to divide a disk into sectors and tracks to receive data.

I/O: Input/Output.

Input: As a noun, data which flows from external devices to a computer. As a verb, to send data from external components.

Input/Output: The software or hardware that exchanges data with the external devices.

Instruction: In a program, the smallest unit of data that can be operated upon, processed by a microprocessor (CPU). In 6502 machine language, an instruction may come in one, two, or three bytes.

Integrated circuit: A small, thin silicon chip into which complicated electronic circuit has been implemented. A single IC can hold from ten to ten thousand discrete electronic components. These ICs are usually packaged in DIP.

Interactive: An operation by the exchange of signals back and forth between computer system and users.

Interface: An exchange of information between one device and another, or the device that makes such exchange possible.

Interpreter: A program that converts high level programs to executable machine codes and executes the interpreted machine codes.

Interrupt: A physical effect that causes the computer to jump to an execute and interrupt service subroutine (a subroutine that requests the CPU to halt its current job and service the subroutine that made the request.) When the computer completed servicing the requesting subroutine, control is returned to where the CPU was previously interrupted.

Inverse video: Changing the original background of characters displayed on the screen, That is, white letters on the black ground will change into black letters on the white ground.

K: Stands for the Greek prefix "kilo", meaning one thousand. In computer usage, it means 2^{10} , or 1,024.

Keystroke: Press a single key or complex keys (as CTRL-D) on the MPF-III's keyboard.

Kilobyte: K bytes, 1024 bytes.

Language: A computer language is a set of characters or a code that is used to form symbols, words, etc., and the rules for combining these into meaningful communications that can be understood by man and the computer.

Language translator: A system program which reads the program written in a specific program language. It may execute the program directly or convert it to another language (machine language) to execute.

Line: On a screen, a line is a horizontal string of graphic symbols, extending from one end of the screen to the other. In the MPF-III, an input line is a string of up to 256 characters terminated by the control character return.

Load: Put data into storing organization of a computer by way of peripheral storing equipment.

Lock: To prevent files on a disk from being changed or deleted.

Logo: A program language in the use of graphics-displayed capabilities of the computer to instruct school children to design programs.

Low level language: Fundamental computer languages whose structure is suitable for the computer to recognize and process. The assembly language is a low level language.

Low order: The least important, or item with the least value in a set of similar items. The low order bit of a byte is the rightmost bit of the byte.

Low part: In a two-byte address, the low order part is the low order byte of the two-byte address.

Low-resolution graphics: Graphics displayed on the MPF-III's screen, composed of 40 by 48 squares in 16 different colors

Machine Language: The language that can be understood by the computer. Basically, it comes in binary. Sets of binary values are grouped to form operation codes (opcode) that specify the operations to be performed by the computer. The opcodes operate upon, or sometimes without, operands (represented also by binary values). An instruction may consist of an opcode and an operand, it may also be an opcode.

Main memory: A memory unit in the computer system. The processor may input and output its contents directly.

Memory address: A value assigned to a memory location. An address only corresponds to a single memory location. The address of a memory location may be expressed in hexadecimal or decimal. For the MPF-III, an address is a two-byte value.

Memory location: The smallest unit of data that can be processed by the computer. It is also the smallest unit of information of a memory map to which the computer can refer. A memory location corresponds with a specific address and a unique value.

Memory map: The set of all memory locations which the microprocessor can address directly. It is used as a graphic representation of a computer system's memory.

Menu: A list of functions which displayed on the screen for users to choose.

Microcomputer: A computer whose CPU is a microprocessor.

Microprocessor: An IC that understands and executes machine language programs.

Mnemonic: A set of symbols designed to help memorize data that is difficult to remember. In assembly language, each instruction is represented by a mnemonic.

Mode: A condition or set of conditions under which a set of rules apply.

Modem: A peripheral equipment. The computer may transmit and receive messages by telephone lines.

Monitor: 1) A program that supervises and services all user's programs. It enables a programmer to operate the computer with assembly programs. 2) a closed-circuit TV receiver.

Monitor program: A system program in the MPF-III, which is used to examine or change directly the contents in the main memory.

Multiplexer: An electronic circuit which has many data inputs, several selector inputs, and one output.

Network: Connect many individually controlled computer by hardware and software.

Nibble: Half of a byte, or four bits.

Opcode: A machine language instruction.

Operating system: A software system to combine information and function of the computer to facilitate users in application programs execution.

Or: A binary function whose output is true or "on" if both or one input is true ("on").

Output: As a noun, data generated by the computer whose destination is the outside world. As a verb, the process of generating such a data.

Page: 1) A unit of measurement for the quantity of memory. A page of memory holds 256 bytes. 2) A full screen of information on a video display.

Parallel interface: An interface which transmits 8 bits through different wires or currents at the same time.

Parallel printer: A printer which receives output information transmitted by way of parallel interface and then print it.

Pascal: 1) A famous French scientist. 2) A programming language.

PC board: Printed circuit board.

Peripheral device: Devices connected to the computer. Most peripherals are input and/or output devices.

Personal computer: Computers with memory, languages, and peripherals which are suitable for use in a home, office, or school.

Pin-out: A description of the function of each pin on an IC.

Printed circuit board: A board of fiberglass or epoxy on which a thin layer of metal has been applied, and then etched away to form traces. Electronic components can be attached to the traces for exchanging signals. Small PC boards are usually called cards.

Processor: A hardware unit used to execute instructions written in machine language and store results in the main memory.

Program: A sequence of instructions created for performing a specific task.

PROM: Programmable Read Only Memory. It is a ROM whose contents can be altered by electrical means. Information stored in PROM remains unchanged when the power is turned off. The contents in PROMs can be erased by ultraviolet light and be reprogrammed.

Prompt: A special symbol which indicates the beginning of an input line.

Power supply: A hardware unit used to convert the received power to the power adequate for other hardware units.

RF modulator: Radio-frequency modulator.

RAM: Random Access Memory, the memory IC to and from which data can be written and retrieved freely. The contents of RAM will disappear as soon as the power is turned off.

Read: Input data from outer apparatus (e.g, disk) to memory unit, or input data to processor from objects (keyboard or main memory) other than the processor.

ROM: Read Only Memory, the memory IC in which data is stored permanently and can not be altered. The data in ROM remains intact even if the power is turned off. ROMs are usually used to store important programs or data such as the monitor program. Data in ROMs is implemented there in the manufacturing process.

Reference: 1) The source of information, such as reference books. 2) As a verb, the action of accessing a memory location.

Return: To exit a subroutine and go back to the program which calls it.

Routine: A part of a program, used to complete a specific part of the program.

Run: Follow and execute instructions in order.

Save: Store information in the main memory in a peripheral installation equipment.

Scan line: A single beam of a cathode beam across the inner surface of a cathode ray tube.

Schematic: A diagram which illustrates the electrical interconnections and the circuitry of an electronic device.

Scroll: To move all the text on a screen up one line.

Serial interface: An interface which transmits data by a single wire or current.

Serial printer: A printer which receives message by way of serial interface and print it.

Soft switch: A switch, usually a toggle type (on/off) switch, controlled by the software.

Software: Programs that request the hardware to do something.

Stack: An area in memory used to temporarily store data. Data is stored in a stack first-in last-out. The data which is first stored in the stack is retrieved last. A programmer stores (PUSH) data onto the stack, and retrieves (POP) it from the stack.

Strobe: A selection signal which indicates the occurrence of a specific event.

Subroutine: A program that can be executed, using the CALL instruction.

Syntax: The rules governing sentence structure in a language.

System program: A program which enables the computer to execute common functions, such as operating system or language translator program.

Telecommunication: To transmit messages by telephone lines in long distance.

Television receiver: A display apparatus, which receives wireless electrical radio signals. It may be combined with an RF modulator in order to be used as a display device for MPF-III.

Text: A collection of characters.

Text file: A file which stores data in characters style.

Text window: Indicating an area of the MPF-III's screen, in which text may be displayed and scrolled.

Toggle switch: An on/off switch which has two states -- either on or off.

Trace: Connected places on a PC board.

User interface: Rules that the computer may use to communicate with users.

Video: 1) Any information that is displayed on a cathode ray tube. 2) Anything that is visible.

Video monitor: An apparatus which receives video signals transmitted by the system directly connected, but not the wireless radio signals.

Viewport: One part or a whole screen which displays parts of data of an application program in operation:

Warm-start: To re-initialize a computer system after a programmer lost control of a program or the operating system.

Window: The segmented portion of a screen is called a window within which information is displayed.

Word processor: An application program used to modify and produce text.

Wraparound: An automatic, continuous feature for word processing on a screen or a printer. (Words may automatically skip from the end of a line to the beginning of the next line.)

Write: Transmit data from the computer to its optional parts (disk drive, printer, or modulator). To computer itself, it is to transmit from its processor to any other apparatus (main memory).

Write enable notch: It is on the edge of a disk jacket so that data may be written into disks. If there is no notch, we can only read the data out, but not write in.

Write protect: The write protect tabs prevent new data from being written in the disks.

Write protect tab: The tab used to stick on the write enable notch so that the data may not be written in the disks.

Appendix B

The 6502 Instruction

ADC	Add Memory to Accumulator with Carry	LDA	Load Accumulator with Memory
AND	"AND" Memory with Accumulator	LDX	Load Index X with Memory
ASL	Shift Left One Bit (Memory or Accumulator)	LDY	Load Index Y with Memory
BCC	Branch on Carry Clear	LSR	Shift Right one Bit (Memory or Accumulator)
BCS	Branch on Carry Set	NOP	No Operation
BEQ	Branch on Result Zero	ORA	"OR" Memory with Accumulator
BIT	Test Bits in Memory with Accumulator	PHA	Push Accumulator on Stack
BMI	Branch on Result Minus	PHP	Push Processor Status on Stack
BNE	Branch on Result not Zero	PLA	Pull Accumulator from Stack
BPL	Branch on Result Plus	PLP	Pull Processor Status from Stack
BRK	Force Break	ROL	Rotate One Bit Left (Memory or Accumulator)
BVC	Branch on Overflow Clear	ROR	Rotate One Bit Right (Memory or Accumulator)
BVS	Branch on Overflow Set	RTI	Return from Interrupt
CLC	Clear Carry Flag	RTS	Return from Subroutine
CLD	Clear Decimal Mode	SBC	Subtract Memory from Accumulator with Borrow
CLI	Clear Interrupt Disable Bit	SEC	Set Carry Flag
CLV	Clear Overflow Flag	SED	Set Decimal Mode
CMP	Compare Memory and Accumulator	SEI	Set Interrupt Disable Status
CPX	Compare Memory and Index X	STA	Store Accumulator in Memory
CPY	Compare Memory and Index Y	STX	Store Index X in Memory
DEC	Decrement Memory by One	STY	Store Index Y in Memory
DEX	Decrement Index X by One	TAX	Transfer Accumulator to Index X
DEY	Decrement Index Y by One	TAY	Transfer Accumulator to Index Y
EOR	"Exclusive-Or" Memory with Accumulator	TSX	Transfer Stack Pointer to Index X
INC	Increment Memory by One	TXA	Transfer Index X to Accumulator
INX	Increment Index X by One	TXS	Transfer Index X to Stack Pointer
INY	Increment Index Y by One	TYA	Transfer Index Y to Accumulator
JMP	Jump to New Location		
JSR	Jump to New Location Saving Return Address		

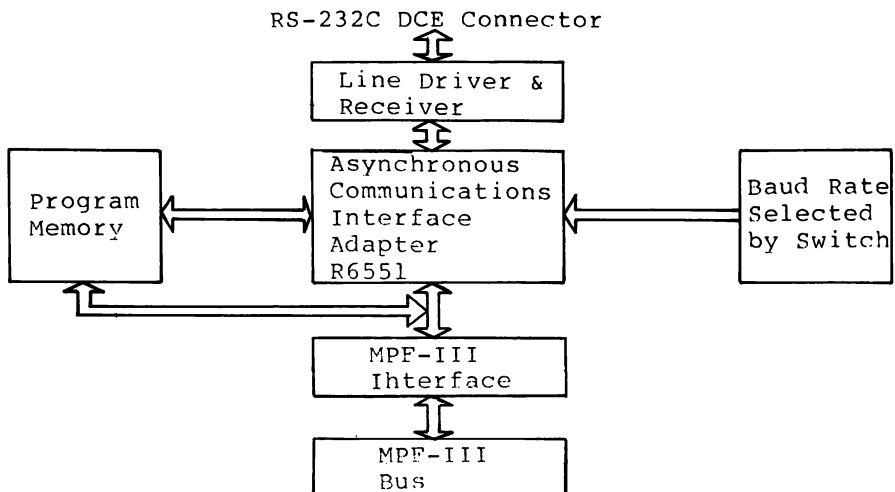
Appendix C

Hardware Structure of RS-232 Interface

The MPF-III is given the capability for Industry Standard Asynchronous Serial Data Transmission via the RS232 interface card.

The RS232 interface card is a programmable circuit board with a core of R6551 ACIA (Rockwell's Asynchronous Communication Interface Adapter). With the RS232 interface card, the MPF-III can be connected to any facilities with the same interface card. It can also communicate with other computer systems through MODEM.

Below is the block diagram showing the primary function units of the RS232 interface card. On the next page, a description of each unit is given.



F1	20	2C	38	44	50	50	50	50
F2	21	2D	39	45	51	51	51	51
F3	22	2E	3A	46	52	52	52	52
F4	23	2F	3B	47	53	53	53	53
F5	24	30	3C	48	54	54	54	54
F6	25	31	3D	49	55	55	55	55
F7	26	32	3E	4A	56	56	56	56
F8	27	33	3F	4B	57	57	57	57
F9	28	34	40	4C	58	58	58	58
F10	29	35	41	4D	59	59	59	59
F11	2A	36	42	4E	5A	5A	5A	5A
F12	2B	37	43	4F	5B	5B	5B	5B

KEY	ALT/ ↑ / CTRL							
	000	001	010	011	100	101	110	111
←	88	88	88	88	88	88	88	88
TAB	89	89	89	89	89	89	89	89
↵	8D	8D	8D	8D	8D	8D	8D	8D
ESC	9B	9B	9B	9B	9B	9B	9B	9B
SPACE	AO	AO	AO	AO	AO	AO	AO	AO
' "	A7	A7	A2	A2	A7	A7	A2	A2
, <	AC	AC	BC	BC	AC	AC	BC	BC
- _	AD	AD	DF	9F	AD	AD	DF	9F
. >	AE	AE	BE	BE	AE	AE	BE	BE
/ ?	AF	AF	BF	BF	AF	AF	BF	BF
0)	BO	Bo	A9	A9	5C	5C	5C	5C
1 !	B1	B1	A1	A1	5D	5D	5D	5D
2 @	B2	80	CO	80	5E	5E	5E	5E
3 #	B3	B3	A3	A3	5F	5F	5F	5F
4 \$	B4	B4	A4	A4	60	60	60	60
5 %	B5	B5	A5	A5	7B	7B	7B	7B
6 ^	B6	9E	DE	9E	7C	7C	7C	7C
7 &	B7	B7	A6	A6	7D	7D	7D	7D
8 *	B8	B8	AA	AA	7E	7E	7E	7E
9 (	B9	B9	A8	A8	7F	7F	7F	7F
: :	BB	BB	BA	BA	BB	BB	BA	BA
= +	BD	BD	AB	AB	BD	BD	AB	AB
[{	DB	9B	FB	9B	DB	9B	FB	9B
\	DC	9C	FC	9C	DC	9C	FC	9C
DEL	FF	FF	FF	FF	FF	FF	FF	FF

KEY	ALT / ↑ / CTRL							
	000	001	010	011	100	101	110	111
] }	DD	9D	FD	9D	DD	9D	FD	9D
、 ~	E0	E0	FE	FE	E0	E0	FE	FE
A	E1	81	C1	81	61	61	61	61
B	E2	82	C2	82	62	62	62	62
C	E3	83	C3	83	63	63	63	63
D	E4	84	C4	84	64	64	64	64
E	E5	85	C5	85	65	65	65	65
F	E6	86	C6	86	66	66	66	66
G	E7	87	C7	87	67	67	67	67
H	E8	88	C8	88	68	68	68	68
I	E9	89	C9	89	69	69	69	69
J	EA	8A	CA	8A	6A	6A	6A	6A
K	EB	8B	CB	8B	6B	6B	6B	6B
L	EC	8C	CC	8C	6C	6C	6C	6C
M	ED	8D	CD	8D	6D	6D	6D	6D
N	EE	8E	CE	8E	6E	6E	6E	6E
O	EF	8F	CF	8F	6F	6F	6F	6F
P	F0	90	D0	90	70	70	70	70
Q	F1	91	D1	91	71	71	71	71
R	F2	92	D2	92	72	72	72	72
S	F3	93	D3	93	73	73	73	73
T	F4	94	D4	94	74	74	74	74
U	F5	95	D5	95	75	75	75	75
V	F6	96	D6	96	76	76	76	76
W	F7	97	D7	97	77	77	77	77
X	F8	98	D8	98	78	78	78	78
Y	F9	99	D9	99	79	79	79	79
Z	FA	9A	DA	9A	7A	7A	7A	7A

Appendix E

A Summary of Built-in Subroutines

Here is a list of useful subroutines in the MPF-III's Monitor. To use these subroutines from machine-language programs, store data into the specified memory locations or 6502 registers as required by the subroutine and execute a JSR to the subroutine's starting address. After the subroutine performs its function, it returns with 6502's registers changed as described.

BEEP Output a bell character \$FF3A

BEEP writes a bell (CTRL-G) character to the current output device. It leaves the accumulator holding \$0.

CONIN Get an input character \$FD0C

CONIN is the character input subroutine. It places a blinking cursor on the display at the cursor position and jumps to the subroutine whose address is stored in KSW (locations \$38 and \$39), usually the standard input subroutine KBDIN, which returns with a character in the accumulator.

CONOT1 Output to screen \$FDF0

CONOT1 displays the character in the accumulator on the MPF-III'S screen at the current output cursor position and advances the output cursor. It places the character using the setting of the Normal/Inverse location. It handles the control characters RETURN, linefeed, backspace, and bell. It returns with all registers intact.

CONOUT Output a character

\$FDED

CONOUT calls the current character output subroutine. The character to be output should be in the accumulator. CONOUT calls the subroutine whose address is stored in CSW (locations \$36 and \$37), which is usually the standard character output CONOUT1.

CREOL Clear to end of line

\$FC9C

CREOL clears a text line from the cursor position to the right edge of the window. CREOL destroys the contents of A and Y.

CREOL1 Clear to end of line

\$FC9E

CREOL1 clears a text line to the right edge of the window, starting at the location given by base address BAS1 indexed by the contents of the Y register. CREOL1 destroys the contents of A and Y.

CREOP Clear to end of window

\$FC42

CREOP clears the text window from the cursor position to the bottom of the window. CREOP destroys the contents of A and Y.

CRPAGE Home cursor and clear

\$FC58

CRPAGE clears the display and puts the cursor in the home position: the upper left corner of the screen.

CRTV Clear the low-resolution screen

\$F832

CRTV clears the low-resolution graphics display to black. If you call CRTV while the video display is in text mode, it fills the screen with inverse-mode at-sign(@) characters. CRTV destroys the contents of A and Y.

CRTVTOP Clear the low-resolution screen

\$F836

CRTVTOP is the same as CRTV (above), except that it clears only the top 40 rows of the low-resolution display.

DELAY Delay**\$FCA8**

DELAY delays for a specific amount of time, then returns to the program that called it. The amount of delay is specified by the contents of the accumulator. With A the contents of the accumulator, the delay is $1/2(26+27A+5A^2)$ microseconds. DELAY returns with the accumulator zeroed and the X and Y registers undisturbed.

GETWORD Get an input character or ESC code**\$FD35**

GETWORD is an alternate input subroutine that gets characters or ESC codes from the standard input subroutine.

HORMAP Draw a horizontal line of blocks**\$F819**

HORMAP draws a horizontal line of blocks of the color set by STDCOL on the low-resolution graphics display. Call HORMAP with the vertical coordinate of the line in the accumulator, the leftmost horizontal coordinate in the Y register, and the rightmost horizontal coordinate in location \$2C. HORMAP returns with A and Y destroyed, and X intact.

INCOL Increment color by 3**\$F85F**

INCOL adds 3 to the current color (set by STDCOL) used for low-resolution graphics.

INVERSE Set inverse mode**\$FE80**

INVERSE sets the display format to inverse. CONOT1 will then display all output characters as black dots on a white background. The Y register is set to \$3F, all others are unchanged.

KBDIN Read the keyboard**\$FD1B**

KBDIN is the keyboard input subroutine. It reads the MPF-III's keyboard, waits for a keypress, and randomizes the random number seed at \$4E-\$4F. When a key is pressed, KBDIN removes the blinking cursor from the display and returns with the keycode in the accumulator.

LOAD Read a record from a cassette **\$FEFD**

LOAD reads a series of tones at the cassette input port, converts them to data bytes, and stores the data in a specified range of memory locations. Before calling **LOAD**, the address of the first bytes must be in A1 (\$3C-\$3D) and the address of the last byte in A2(\$3E-3F).

MAPPING Plot on the low-resolution screen **\$F800**

MAPPING puts a single block of the color value set by **STDCOL** on the low-resolution display screen. The block's vertical position is passed in the accumulator, its horizontal position in the Y register. **MAPPING** returns with the accumulator destroyed but X and Y intact.

MVDATA Move a block of memory **\$FE2C**

MVDATA copies the contents of memory from one range of locations to another. This subroutine is the same as the **MOVE** command in the Monitor, except it takes its arguments from pairs of locations in memory, low-byte first. The destination address must be in A4 (\$42-\$43), the starting source address in A1(\$3C-\$3D), and the ending source address in A2(\$3E-\$3F) when your program calls **MVDATA**.

NORMON Set Normal mode **\$FE84**

NORMON sets the display format to normal. **CONOT1** will then display all output characters as white dots on a black background. On return, the Y register is set to \$FF, all others are unchanged.

OUT8D1 RETURN with clear **\$FD8B**

OUT8D1 clears the screen from the current cursor position to the edge of the text window, then calls **OUT8DH**.

OUT8DH Generate a RETURN **\$FD8E**

OUT8DH sends a RETURN character to the current output device.

OUTAX Print A and X in hexadecimal \$F941

OUTAX prints the contents of the A and X registers as a four-digit hexadecimal value. The accumulator contains the first byte output, the X register contains the second. On return, the contents of the accumulator are destroyed.

OUTBYTE Print a hexadecimal byte \$FDDA

OUTBYTE outputs the contents of the accumulator in hexadecimal on the current output device. The contents of the accumulator are destroyed.

OUTERR Print ERR \$FF2D

OUTERR sends the word ERR, followed by a bell character, to the standard output device. On return, the accumulator is destroyed.

OUTHEY Print a hexadecimal digit \$FDE3

OUTHEY prints the lower nybble of the accumulator as a single hexadecimal digit. On return, the accumulator is destroyed.

OUTSP1 Print many blank spaces \$F94A

OUTSP1 outputs from 1 to 256 blanks to the standard output device. Upon entry, the X register should contain the number of blanks to be output. If X=\$00, then OUTSP1 will output 256 blanks.

OUTSPACE Print 3 spaces \$F948

OUTSPACE outputs three blank spaces to the standard output device. On return, the accumulator usually contains \$A0, the X register contains 0.

PDLRED Read a hand control \$FB1E

PDLRED returns a number that represents the position of a hand control. You pass the number of the hand control in the X register. If this number is not valid (not equal to 0, 1, 2, or 3), strange things may happen. PDLRED returns with a number from \$00 to \$FF in the Y register. The accumulator is destroyed.

RDLIN0 Get an input line **\$FD67**

RDLIN0 is an alternate entry point for **RDLINE** that sends a carriage return to the standard output, then continues into **RDLINE**.

RDLIN1 Get an input line, no prompt **\$FD6F**

RDLIN1 is an alternate entry point for **RDLINE** that does not issue a prompt before it accepts that input line. If, however, the user cancels the input line, either with too many backspaces or with a **CTRL-X**, then **RDLIN1** will issue the contents of location **\$33** as a prompt when it gets another line.

RDLINE Read an input line with prompt **\$FD6A**

RDLINE is the standard input subroutine for entire lines of characters. Your program calls **RDLINE** with the prompt character in location **\$33**; **RDLINE** returns with the input line in the input buffer (beginning at location **\$200**) and the **X** register holding the length of the input line.

RDTV Read the low -resolution graphics screen **\$F871**

RDTV returns the color value of a single block on the low-resolution graphics display. Call it with the vertical position of the block in the accumulator and the horizontal position in the **Y** register. Call it as you would call **MAPPING** (above). The color of the block will be returned in the accumulator. No other registers are changed.

RGSAVE Save all registers **\$FF4A**

RGSAVE stores the contents of the 6502's internal registers in locations **\$45** through **\$49** in the order **A**, **X**, **Y**, **P**, **S**. The contents of **A** and **X** are changed and the decimal mode is cleared.

RGRSTD Restore all registers **\$FF3F**

RGRSTD loads the 6502's internal registers with the contents of memory locations **\$45** through **\$49**.

SAVE Write a record on a cassette

\$FECD

SAVE converts the data in a range of memory to a series of tones at the cassette output port. Before calling SAVE, the address of the first data byte must be in A1(\$3C-\$3D) and the address of the last byte in A2(\$3E-\$3F). The subroutine SAVE generates a ten-second continuous tone as a header, then writes the data followed by a one-byte checksum.

STDCOL Set low-resolution graphics color

\$F864

STDCOL sets the color used for plotting in low-resolution graphics to the value passed in the accumulator.

VERMAP Draw a vertical line of blocks

\$F828

VERMAP Draw a vertical line of blocks of the color set by STDCOL on the low-resolution display. You should call VERMAP with the horizontal coordinate of the line in the Y register, the top vertical coordinate in the accumulator, and the bottom vertical coordinate in location \$2D. VERMAP will return with the accumulator destroyed.

VRDATA Compare two blocks of memory

\$FE36

VRDATA compares the contents of one range of memory to another. This subroutine is the same as the VERIFY command in the Monitor, except it takes its arguments from pairs of locations in memory, low-byte first. The destination address must be in A4 (\$42-\$43), the starting source address in A1 (\$3C-\$3D), and the ending source address in A2 (\$3E-\$3F) when your program calls VRDATA.

Subroutines Used in SG

A number of subroutines are available in the MPF-III SG. To produce melodious music or generate special sound effects, you have to use a JSR to jump to the start address of these subroutines.

- | | |
|------------|--|
| 1. \$A05E | Delay 0.5 second |
| 2. \$A060 | Delay Y*2ms |
| 3. \$A063 | Delay Yms |
| 4. \$A04A | Suppress the three channels of 8912 |
| 5. \$A0E2 | Generate the sound of the machine gun for 0.5 sec |
| 6. \$A105 | Generate the sound of the machine gun for Y*0.5 sec |
| 7. \$A0B3 | Generate ten contiguous sounds of the laser gun |
| 8. \$A0B5 | Generate Y contiguous sounds of the laser gun |
| 9. \$A06C | Generate the sound of the bomb during its falling to the ground |
| 10. \$A06E | The value of 0CCH decides the duration of the sound of the falling bomb. |
| 11. \$A092 | Generate the sound of explosion |
| 12. \$A4B8 | Auto play the music starting with the location defined by (CF) (CE) |
| 13. \$A2DC | X sets the accompaniment (X=1-8)
Accompaniment suppressed when X=0 |
| 14. \$A426 | Select Bell |
| 15. \$A42F | Select Piano |
| 16. \$A41D | Select Xylophone |
| 17. \$A438 | Select Organ |



Multitech INDUSTRIAL
CORP.

OFFICE/ 315 FU HSIN N. RD., TAIPEI, TAIWAN, R.O.C.
FACTORY/ 1 INDUSTRIAL E. RD., III HSINCHU SCIENCE BASED
INDUSTRIAL PARK, HSINCHU, TAIWAN, R.O.C.



Microsoft Research

Microsoft Research is the leading edge of computing research, working on the next generation of computing technologies. Our research is focused on the areas of artificial intelligence, human-computer interaction, and systems. We are currently working on a number of exciting projects, including the development of a new generation of artificial intelligence systems, the creation of a new generation of human-computer interfaces, and the development of a new generation of systems. Our research is funded by the Microsoft Research Foundation, and we are currently seeking talented individuals to join our team. If you are interested in pursuing a career in research, please contact us at research@microsoft.com.